TECHNICAL REPORT · V1.0 · REVISED NOVEMBER 2026 November 2026

Robot Skill Factory

Towards Recursive Self-Improvement in Humanoid Control

Implementation and Evaluation on a Simulated Unitree G1

Di Yang
Contact: dyang06@wm.edu
Code: https://github.com/IoET-y/robot-skill-factory

Abstract

Adding a skill to a humanoid robot usually means collecting data, retraining the controller, checking that earlier behaviors still work, and integrating the result by hand. When one whole-body tracker executes every motion skill, an update made for one request can affect previously acquired skills. We study recursive self-improvement (RSI) in humanoid control through persistent skill acquisition and reuse. Robot Skill Factory integrates skill acquisition and verification with planning and continuous execution. A language-model planner serves a request from a registry of versioned skills or asks the factory for a missing one. The factory probes the current tracker on the requested motion type, trains a new tracker version only when the probe fails, verifies it on held-out performers and on earlier motions, and registers the skill; a regression gate decides whether the new version replaces the current one. The runtime, ORCHESTRA, executes plans as continuous simulation runs that pass control between the tracker, a walking policy, and object-interaction policies. On a simulated Unitree G1 with one RTX 4090, the factory met the acceptance criteria of 30–31 of 37 benchmark requests, against 21 for the same planner with the registry alone; this benchmark ran before the gate was added. In a separate seven-request sequence, the gate rejected an update that reduced completion of a previously acquired skill's plan from 28/32 to 0/32 worlds. A request that needs training reaches its executed plan in 11–12 minutes and the promotion decision in about 18 minutes. Starting from a tracker trained on 127k clips instead of 40, the factory needed training for 7 instead of 11 of 30 new motion types, which cut their total acquisition time by 35%; an acquisition with training took about the same time from either tracker.

1 Introduction

A humanoid that serves open-ended requests will be asked for skills it does not have. Adding one is a sequence of manual steps: an engineer finds motion data, retrains the controller, checks that earlier behaviors still work, and integrates the result into the task stack. With a shared whole-body tracker, every motion skill runs on the same policy, so an update made for one request can affect previously acquired skills.

For recursive self-improvement, acquired capabilities must persist and remain available to later tasks. This requires skill acquisition to be coupled with retention checks, controller versioning, and execution. We investigate this loop using a shared humanoid tracker.

The components exist separately. Whole-body trackers trained in GPU simulation reproduce large motion libraries with one policy [25, 32], and recent ones train in minutes on one GPU [4, 15]. Language-model agents compose humanoid skills from requests but keep the controller frozen [14, 28, 42, 44]. Systems that extend a humanoid controller start each update by human decision [13, 34],

and the closest automated loops retrain a manipulation policy when a task exposes a missing primitive [47, 48], without a shared controller whose update affects every skill.

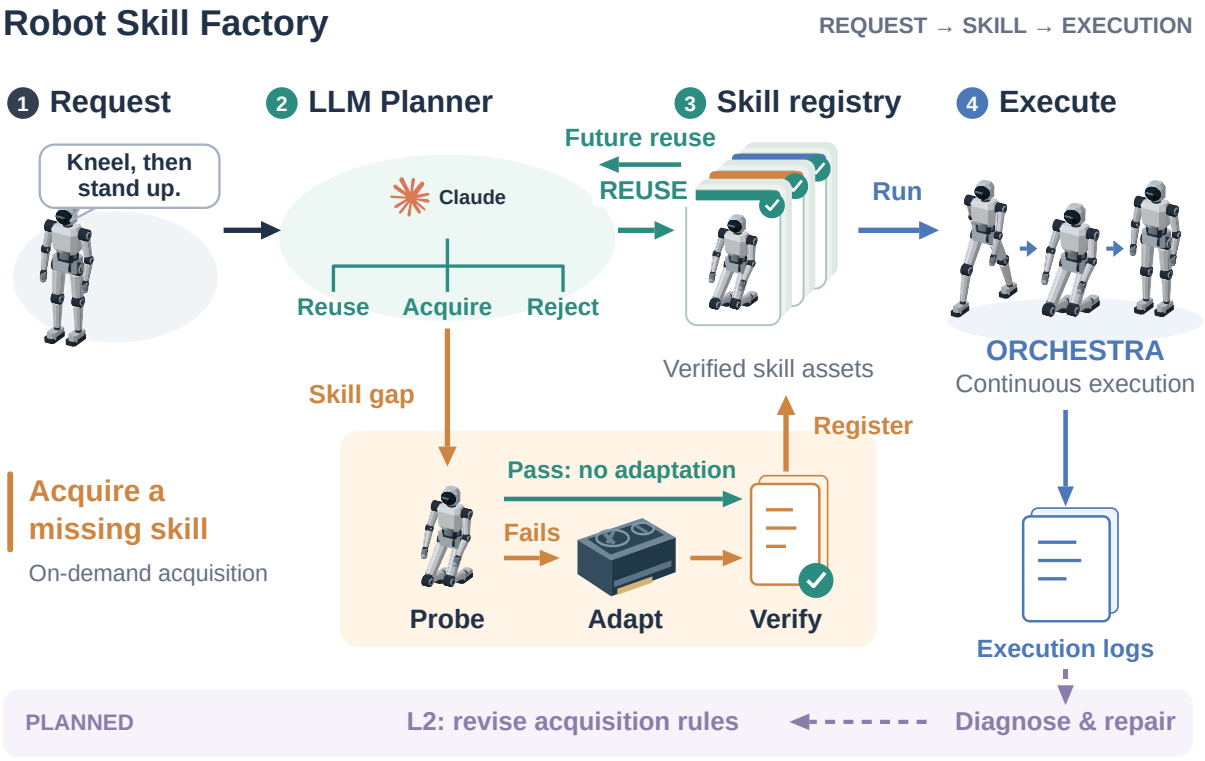


Figure 1. From task requests to reusable robot skills. The planner reuses registered skills or requests new ones; the factory probes, adapts when needed, and verifies them before registration. ORCHESTRA executes the resulting plans. Dashed paths represent planned improvements.

Robot Skill Factory connects these steps (Figure 1). A shared registry holds skills as versioned assets: a reference motion or a policy, the tracker version that verified it, its entry and exit states, and its evaluation record. A language-model planner serves a request from the registry or asks the factory for a missing skill. The factory probes the current tracker on the requested motion type and trains a new tracker version only when the probe fails; it verifies the new version on held-out performers and on regression sets, registers the skill, and passes the version to a regression gate that decides whether it replaces the current tracker. ORCHESTRA, the runtime, executes plans as continuous simulation runs that pass control between the tracker, a walking policy, and object-interaction policies, without resets or writes to the simulation state.

We evaluate the implementation on a simulated Unitree G1 with one RTX 4090. On a benchmark of 37 requests, the factory met the acceptance criteria of 30–31, against 21 for the same planner with the registry alone. In a controlled comparison over three seeds, replaying the training clips of earlier acquisitions reduced forgetting of previously acquired skills, and in a separate seven-request sequence the gate rejected an update that reduced completion of a previously acquired skill's plan from 28/32 to 0/32 worlds. Zero-shot return on held-out motion types grows with the number of motion types in the training library more than with the number of clips, and levels off at a return of about 0.90 once the library holds about a thousand types and ten thousand clips, after 32k training iterations. The cost of a new skill depends mainly on how often the probe triggers training: an

acquisition with training took 624–655 s and one without 15–20 s, and a tracker trained on 127k clips needed training for fewer new motion types than one trained on 40. For requests that triggered training, verification and promotion took longer than training itself.

This report makes three contributions:

- **Robot Skill Factory**, a system integrating on-demand skill acquisition, a versioned registry, a regression gate for shared tracker updates, and continuous execution across policies (Section 3).
- **An evaluation** of acquisition, retention, cross-request reuse, and regression detection through a request benchmark, controlled replay experiments, and sequential acquisition tests (Section 5), with additional evaluations in Section 7.
- **An efficiency analysis**: how the training library changes zero-shot generalization and how often training is needed, and how the time of an acquisition divides between training, verification, and promotion (Section 6).

2 Related work

Motion learning and continual adaptation. Physics-based motion imitation trains a policy to follow reference motions [40]. On humanoids, BeyondMimic [25] tracks diverse motions on a Unitree G1, SONIC [32] scales one tracker to 700 hours of motion capture, SuperMimic [26], InstantMimic [6], InstantTrack [15], and BlinkMimic [4] train in minutes, Uni-SuperMimic [35] trains on the full BONES-SEED library, and ProtoMotions [43] packages retargeting, training, and deployment. Scale improves tracking of unseen motions [32], with diminishing returns [41], saturation that follows behavior coverage [57], or a larger role for data quality [10], and pretrained trackers learn downstream tasks faster than training from scratch [16, 31]. Trackers are also extended after their first training, by continual expansion [34], refinement on generated motions [54], adapter columns per motion category [13], or warm-started rounds on verified references [59]. In all of these, a person decides when the controller changes and on which data, and none measures how the cost of the next skill changes as skills accumulate.

Skill composition and robot agents. Language models ground plans in a fixed set of skills [1], write programs over control primitives [24], and grow libraries of code skills [46]. For humanoids, HomeBody [14] calls hand-built skills on a real G1, CHOREO [42] sequences trajectory assets with entry and exit descriptors for a frozen tracker, the closest analogue to our skill manifests, InterEvolve [28] evolves reward programs for a frozen behavior model, and Metatron [44] distills a promptable policy whose skills are goal-emitting programs. BAT [2] switches between whole-body controllers online, Switch [19] shows that switching between agile skills in mid-motion is hard for general trackers, and LessMimic [27] evaluates long-horizon interaction without resets. Object interaction adds an object-aware residual to a general tracker [60], contact, scene, object, or force information [5, 22, 50–53, 58], or a walk to a pose computed from the live object pose before an interaction policy takes over [9], the pattern of our interaction steps; HAIC [20] carries a box on a real G1 with proprioception only, and Humanoid Hanoi [17] notes that turning away after a placement can hit the placed boxes. In these systems the controller stays fixed or the skills are built by hand, so the set of skills does not grow from the robot's own requests.

Automated skill acquisition and recursive self-improvement. Agentic systems learn new behaviors by generating tasks and training a policy for each [23, 49], by growing libraries of verified repair strategies [30], or by training proposed skills on a shared backbone [55]. Closest to our loop, InSight [47] detects primitives that a vision-language-action model lacks, collects demonstrations,

and retrains the model while checking base skills, and AdaRSI [48] separates a poor way of calling an executor from a motion the executor cannot make, close to our distinction between apparent gaps and motion gaps. EmbodiRSI [18] applies recursive self-improvement to robot adaptation, one survey grades such claims by levels of evidence [11], and another treats post-deployment self-improvement of humanoids as open [36]; HITCH [29] shows that some capability gaps close by changing how an expert is trained. Table 23 in Section D compares these systems. Robot Skill Factory applies such a loop to a shared humanoid whole-body controller, where every update touches every existing skill, and measures the cost of each acquisition.

3 Robot Skill Factory

Robot Skill Factory has three components around a shared registry (Figure 2): a language-model planner that turns a request into a plan of registered skills, the factory that acquires missing skills, and ORCHESTRA, which executes plans. This section follows one request through them; Section A gives implementation details.

3.1 Skills, tracker versions, and the registry

A *motion clip* is one recorded or retargeted reference motion, and a *motion type* groups all clips of one described motion across performers. A *registered skill* is a registry entry that the planner can use. A motion skill is a window of at most 12 s of one clip together with the tracker version that verified it; an interaction skill is a policy together with its object, a box or a terrain, and the object pose from which it starts. Each skill's manifest also stores its entry and exit states, its provenance, including whether the clip is in the tracker's training data, its data license, and its evaluation record. A tracker may perform motions it was never trained on, and the registry holds far fewer skills than the tracker has training clips: this report uses 38 motion skills and 12 interaction skills, on trackers trained on up to 127k clips.

Whether one skill can follow another depends on the jump between the exit state of the first and the entry state of the second. The *compatibility gap* is the largest of four normalized differences between the two states: the root-mean-square difference of the joint positions divided by $0.1/\sqrt{3}$ rad, the difference in root height divided by 0.01 m, and the differences in roll and in pitch divided by 0.1 rad; heading is excluded. The divisors are the tracker's reset-noise scales during training; the factory uses the gap to rank candidate clips.

All motion skills share one whole-body tracker, InstantTrack [15], a 9.12M-parameter policy trained by backpropagation through a learned world model in mjlab [56] on MuJoCo Warp [8, 45]. Its observation contains no global position and no linear velocity, so joint encoders and an IMU can provide it on hardware. Every training run produces a new version of the network, warm-started from its parent, and records the parent and the clips it trained on, so the training data of any version follow from its chain of warm starts. A registry pointer names the current version, and ORCHESTRA executes every tracking step of a run with one version.

A request that the registry cannot serve exposes one of three kinds of gaps. A *motion gap* closes with motion data, as when the tracker learns to stand up from a kneel. A *capability gap* needs a new observation, scene, or kind of policy, as when the robot must carry a box. An *apparent gap* closes without a new skill, for example by applying an existing skill to a rotated object.

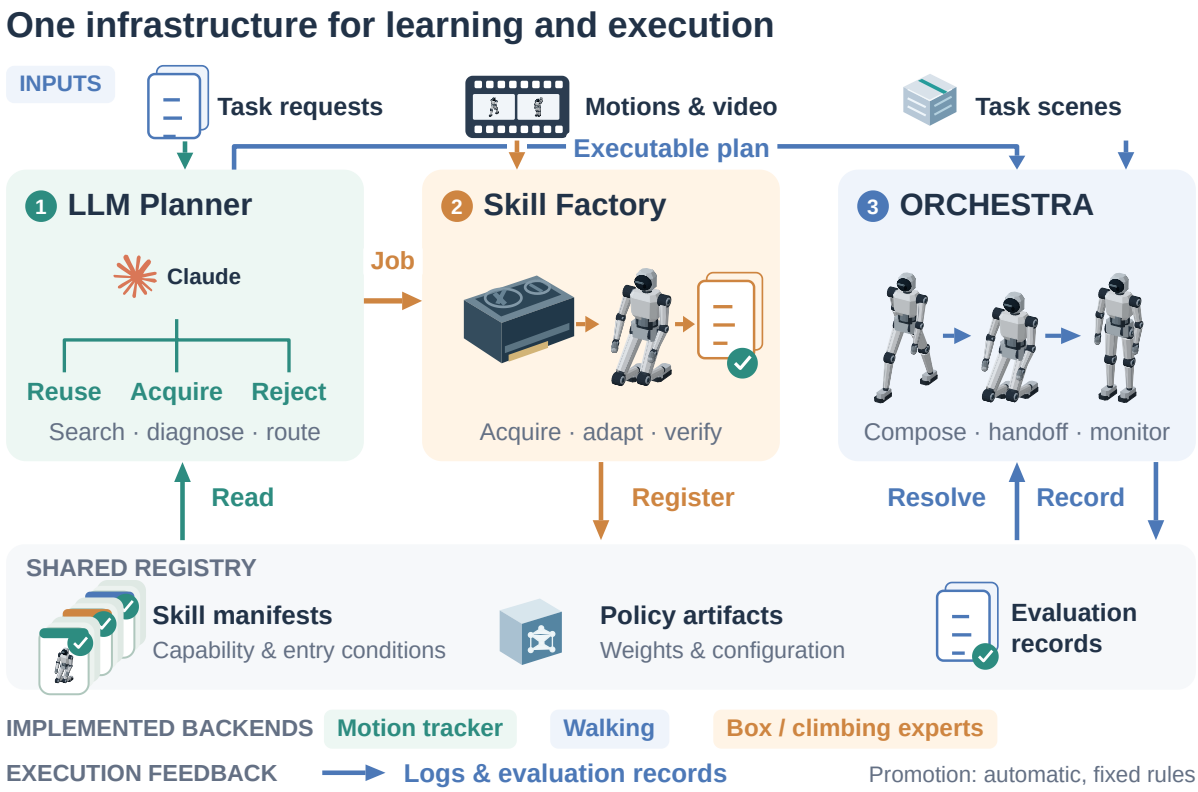


Figure 2. Robot Skill Factory architecture. A shared registry connects the LLM planner, skill acquisition, and ORCHESTRA execution. The lower rows list the policy backends implemented in this report and the execution records.

3.2 Planning

The planner is Claude Haiku 5.5 with tool use. It reads the registry, validates draft plans with `check_plan`, which checks step parameters and whether each step can start where the previous one ends and predicts where the robot and any object end up, and returns a plan or a refusal. When a request needs a skill that the registry lacks, the planner can search the motion library and the robot–object clips and request a motion type or a clip. A request may take up to three planning rounds that request skills, followed by one that may only use registered skills (Algorithm 1 in Section A).

3.3 Acquisition

Sources. BONES-SEED [3] contributes 129,485 G1 motions after conversion and filtering, indexed by description, event labels, and tags and grouped by motion type. Web videos become G1 motions through GEM-X [37], which estimates the person's motion, and soma-retargeter [38], which retargets it to the robot, followed by grounding and filtering. OmniRetarget's G1 data [52] contributes 595 robot–object and robot–terrain clips, for which we generate text descriptions from motion and scene measurements because the data has none.

Probe. For a requested motion type, the factory first probes the current tracker on up to 40 clips of the type, taken from its performers in turn, with 2 rollouts each. The probe passes when no rollout fails and the mean return is at least 0.85; for motions estimated from video, which carry estimation

noise, a lower level can be set (0.70 in Section 7.2). When the probe passes, the factory registers one skill from the probed clips, chosen as described below, without training.

Training and verification. When the probe fails, the factory continues training the current tracker from its full training state for a fixed 0.1 h, on the clips of about 80% of the type's performers together with a replay of 5,000 library clips, three copies of LAFAN1, and the training clips of every earlier acquisition along the tracker's chain of warm starts. It then evaluates the new version on the clips of the held-out performers, on the training clips, and on three regression sets: a sample of the first BONES-SEED shard, the 112 held-out motion types, and LAFAN1.

Registration. The factory registers one skill per acquisition. The candidates are the evaluated clips of the type with no failed rollout and a skill window of at most 12 s; no return threshold applies at this step. Among them, the factory prefers the clip with the smallest compatibility gap to the exit state of a registered skill, adding 10 when the foot contacts differ, and then the highest return, so that the new skill composes with the registry. Registration is determined by these clip-level criteria; the new skill then runs alone in ORCHESTRA in 32 randomized worlds, and the result is logged for evaluation.

Promotion. A trained tracker is a candidate. In the loop, after the request's plan has run with it, a regression gate with fixed tolerances decides whether the candidate becomes the current tracker (Section A). The gate runs both trackers on the same worlds and clips: every registered plan and every registered motion skill alone in ORCHESTRA, the regression sets, and the held-out performers of earlier acquisitions and of the new motion type. If every check passes, the factory switches the registry pointer and logs the decision with the candidate's lineage; otherwise it keeps the current tracker and withdraws every skill registered with a tracker version trained since the current one, including intermediate versions from the same request.

Interaction policies. For a robot–object or robot–terrain clip, the factory trains a policy of a predefined family on mjlab's tracking task, a box-carrying policy or a climbing policy (Section 7.1). It trains in stages and runs the request's plan after each stage, until the new skill succeeds in at least 90% of the worlds or the stages run out.

Records. Every acquisition records its cost, as wall-clock time from request to executed plan, GPU time, training iterations, and API cost, and its effect, as the return and failed rollouts of the new type on held-out performers and the change on the regression sets; every promotion decision is logged with the results of both trackers.

3.4 Execution

ORCHESTRA executes a plan as one continuous run in each of N parallel worlds ($N = 32$ in this report). A run places the robot and any objects once and afterwards evolves only through the robot's actions: there is no reset and no direct write to the state of the robot or of an object. The reference pointer, the index of the reference frame that the tracker follows, is not physical state; it may change, and every change is counted. Running whole plans in this way exposes failures that per-skill evaluation with resets hides, such as a skill that fails only when it starts from the end state of the previous one.

ORCHESTRA compiles the motion steps into one reference timeline with short blended transitions and passes control between three kinds of policies: the shared tracker for motion skills, a walking policy (mjlab's velocity-tracking task) for walking, and interaction policies for interaction steps. Handoffs happen at standing poses, with the reference rotated to the robot's current heading. A

`walk_to` step closes the loop on position with a pure-pursuit controller. An `interact` step moves the skill's reference, robot and object, to where the object actually is, walks the robot to the moved start pose, hands over to the interaction policy, and afterwards backs the robot away from the object before it turns.

After every simulation step, the first monitor that fires stops a world and records the step, the skill, the time, and the values: a fall, by the active policy's own termination test; a body position error above 0.20 m, the root-mean-square position error of 14 tracked bodies relative to the reference moved to the robot's position and heading, averaged over the last second; a heading error above 45°, averaged in the same way; or an object more than 0.5 m from its target.

4 Experimental setup

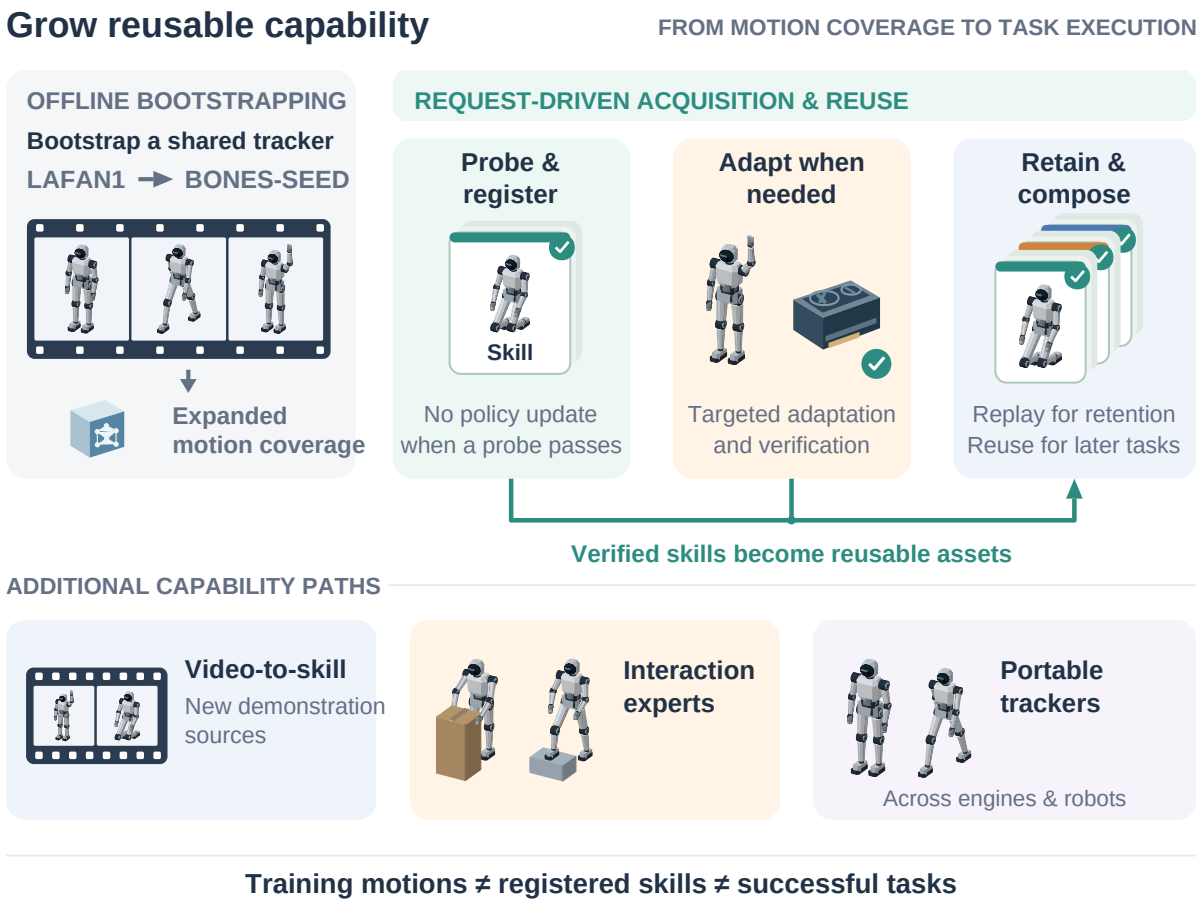


Figure 3. Growing reusable robot capabilities. Offline training expands motion coverage, while request-driven acquisition verifies and registers skills for reuse and composition. Video, interaction experts, deployment to other physics engines, and a tracker trained for a second robot extend the system.

Robot and compute. The robot is a simulated Unitree G1 with 29 actuated joints in mjlab on MuJoCo Warp; the policies run at 50 Hz and the physics at 200 Hz. The planner, every acquisition in the loop, and every plan execution ran on one RTX 4090 with 24 GB; the scaling studies of Section 6 and the five climbing policies of Section 7.1 were trained on A800 GPUs with 80 GB. The planner calls `claude-haiku-5-5` with adaptive thinking at medium effort, up to 16,000 output tokens per call,

and up to 12 calls per planning round.

Data. LAFAN1 contributes 40 clips retargeted to the G1 [33], which are in the training data of every tracker the factory produces. Of the 129,485 BONES-SEED motions, 112 motion types (2,541 clips) are held out from all training, except where an acquisition trains on one of them, and the remaining 126,944 clips form 13 training shards of about 9,770 clips. When the factory trains on a motion type, it holds out about 20% of the type's performers for verification.

Tracker versions. The shared tracker is first trained offline, and the factory then adapts it on request (Figure 3). T_0 was trained from scratch on the LAFAN1 clips for 0.1 h; in this run and in a second seed, no clip failed for three consecutive checkpoints after 4.5–5.0 min. $T_1, \ldots, T_{13}$ each add one BONES-SEED shard, warm-started from the previous version for 0.1 h with three copies of LAFAN1, 88 min in total. From T_0 to T_{13}, return on the 112 held-out motion types rose from 0.827 to 0.901 and failed rollouts fell from 2.9% to 0.8%; 14 of the 15 remaining failures were on one type, standing up from a kneel (Table 6 in Section C). T_K results from the stand-up acquisition of Section 5.2; B_1, B_2, B_4, and B_5 from training runs in a five-request sequence that starts from T_K (Section 5.3); and T_V from a motion acquired from video (Section 7.2). T_V is the current tracker of the released registry; Figure 8 in Section B draws the lineage.

Metrics. A tracker's *return* on a clip is InstantTrack's normalized tracking return: per step, a weighted sum of Gaussian kernels of the position, orientation, and velocity errors of 14 tracked bodies and of the root, minus penalties on action changes and joint-limit violations, summed over the clip and divided by its largest possible value, so that 1 means perfect tracking. A rollout *fails* when the robot's root height or tilt departs too far from the reference (Section B); the steps after a failure score zero. Reported returns are means over clips and rollouts, each clip weighted equally. Tracker evaluations start at each clip's first frame without domain randomization, with 2 rollouts per BONES-SEED clip and 4 per LAFAN1 or video clip. Results on clips in a tracker's training data are reported separately from results on new clips.

Plans run in ORCHESTRA in 32 worlds with the tracker's training-time domain randomization: friction, center of mass, joint offsets, and observation noise and delay. Every plan execution reported had 0 resets and 0 state writes, and figures show world 0, which is not selected. We report three outcome measures. *Plan completion* counts the worlds that finish a plan without a monitor stop. *Task success* adds a check of the request's own goal where one exists, such as a box ending within 10 cm of its target or a climbing skill's end test. *Benchmark acceptance* (Section 5.1) requires completion in at least 90% of the worlds and steps that match the components of the request. Results come from one training seed unless stated otherwise.

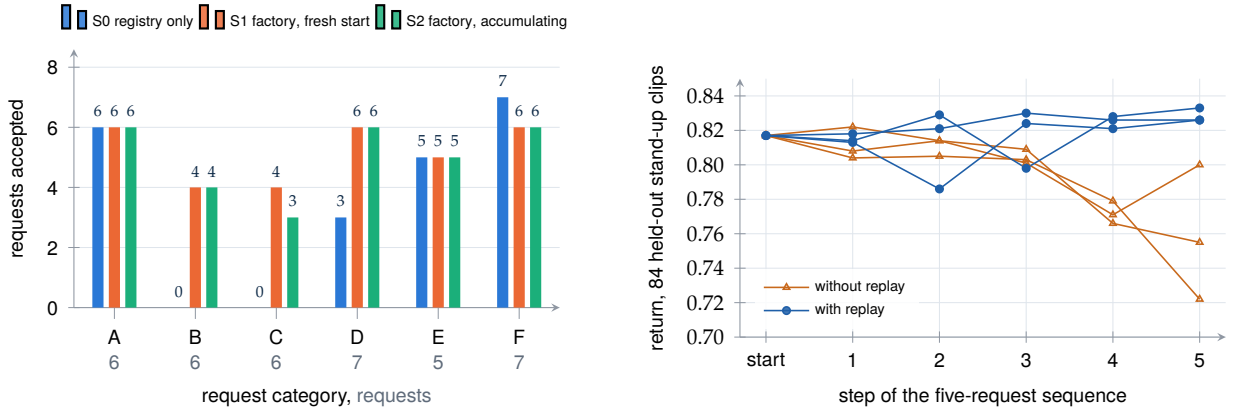
5 System evaluation

We first measure the whole system on a benchmark of requests and then test what skill-level RSI requires of it: whether an acquisition closes the gap that a request exposes (Section 5.2), whether new skills persist through later updates and whether the gate rejects updates that damage them (Section 5.3), and whether acquired skills serve later requests (Section 5.4).

5.1 A system-level benchmark

rsf-bench v0 has 37 requests in six categories: an existing skill (A), a new motion type that passes the probe (B), a new type that fails it (C), compositions, three of which need skills that earlier requests of the set acquire (D), walking and interaction experts (E), and requests that the system

should decline (F). A request of categories A–E is accepted when its plan completes in at least 90% of 32 randomized worlds and contains, in order, steps that match the components of the request; a request of category F is accepted when no plan is executed. Three systems ran the requests in the same order from T_V: S0, the planner with the registered skills only; S1, the factory, with every request starting from T_V and the initial registry; and S2, the factory with accumulation, in which every request starts from the tracker and registry that the previous request left. The benchmark ran before the promotion gate was added, so in S2 every trained tracker served the next request directly.



(a) rsf-bench v0 by request category.

(b) Controlled replay, three training seeds.

Figure 4. System-level results. (a) rsf-bench v0: requests accepted by category for the planner with the registry alone (S0), the factory with every request starting from the same tracker and registry (S1), and the factory with accumulation (S2), before the promotion gate was added; gray numbers give the requests per category. A: an existing skill; B: a new motion type that passes the probe; C: one that fails it; D: a composition; E: walking and interaction experts; F: a request to decline. One run per system. (b) Controlled replay: return of the earliest acquisition, standing up from a kneel, on its 84 held-out clips after each step of a five-step sequence from T_K, with and without replay of earlier acquisitions; one line per training seed.

S0 met the acceptance criteria of 21 requests, S1 of 31, and S2 of 30 (Figure 4a; Table 19 in Section C): 14, 25, and 24 of the 30 requests that call for execution, and 7, 6, and 6 of the 7 that should be declined. Acquisition added new motion types (B: 0 to 4 of 6; C: 0 to 3–4 of 6) and the compositions that need them (D: 3 to 6 of 7), at about twice the API cost of S0. Accumulation changed cost more than acceptance: the lunge followed by the push-up position (D7) took 688 s in S1, which acquired both skills again and retrained the push-up, and 27 s in S2, which reused them. After the run, the final tracker of S2 re-ran all 30 plans that S2 had executed and completed 959 of 960 worlds. Only three of the 37 requests depend on earlier ones, and each system ran once. Most failures came from the planner approximating a request with registered skills instead of requesting the motion type the request names.

5.2 An acquisition in detail

Table 1 follows four requests about kneeling and standing. For requests 1 and 2, the planner found clips that contain the whole sequence, a prayer and a bow, which T_{13} already performed, and the factory registered them without training. For request 3, the library's only stand-up-from-kneel motion type failed the probe. The factory trained on 55 of the type's 69 performers for 6.6 min, which produced T_K. On the 14 held-out performers, return rose from 0.734 to 0.817 and failed rollouts fell from 6 of 168 to 0; earlier motions and LAFAN1 kept their return, and the registry's plans still completed in 128 of 128 worlds (Table 7 in Section C). Asked request 2 again, the planner

composed the new kneel and stand-up skills in 28 s for $0.007, where it had first approximated the request with a bow. The composition worked because registration prefers the clip whose entry state is closest to the exit state of a registered skill: the registered stand-up clip was the closest of 124 candidates to the exit of the registered kneel.

Table 1. Four requests served by the loop, in order, each executed in 32 randomized worlds. Requests 1 and 2 registered motions that T_{13} already performed; request 3 trained T_{13} into T_K; request 4 repeats request 2 after request 3. Probe: mean return and failed rollouts of the current tracker on the requested motion type.

#	Request	Motion types requested	Factory path	Executed	Time	API
1	"Kneel down on one knee, stay there for two seconds, then stand back up."	`prayer_R`	probe passes (0.946, 0/20); registered	32/32	49 s	$0.011
2	"Kneel down on one knee with both arms hanging at your sides, stay down for two seconds, then stand up again."	`my_lord_R`, `kneeling_start_R`	both probes pass; registered	32/32	94 s	$0.021
3	"The robot starts out kneeling on one knee. Have it stand up, then walk two meters forward."	`kneeling_stop_R`	probe fails (0.670, 9/80); 6.6 min of training; T_K registered	32/32	11 min 46 s	$0.010
4	as request 2	none	composition only	32/32	28 s	$0.007

Training increased held-out return from 0.734 to 0.817; plan completion was unchanged in the before-and-after tests. Run again in the same randomized worlds with T_{13} and with T_K, three seeds of 32 worlds each, the executed plan of request 3 completed all 96 runs with both trackers, and plans of the same shape with 28 other stand-up clips completed in 415 of 448 worlds with either tracker; the plan's largest one-second average body error was 11.2 cm before the training and 12.9 cm after (Table 8 in Section C). In the four training steps of the sequence in Section 5.3, the executed plans also completed in 96 of 96 runs with the tracker from before the training, and their body and heading errors fell, most for a locking sequence whose largest one-second average body error fell from 19.2 to 6.2 cm. In the benchmark, 7 of the 10 requests that trained completed their executed plans equally well with the tracker from before the training. The probe measures tracking quality across a motion type; it does not test whether a request's plan needs that quality.

5.3 Retention across acquisitions

When the factory trains the tracker, it replays a sample of the library, three copies of LAFAN1, and the training clips of every earlier acquisition along the tracker's chain of warm starts. In a sequence of five requests from T_K, each asking for a motion that was not yet registered, the factory served all five, four of them by training: every executed plan completed in 32 of 32 worlds, and the last tracker, B_5, completed all plans of the sequence and the registry in 288 of 288 worlds (Table 9 in Section C). A request that trained took 11.2–11.3 min to its executed plan.

To isolate replay, we repeated the five acquisitions from T_K, 2,000 iterations each, with the same data except that the training clips of earlier acquisitions were either replayed or replaced by as many random library clips, three seeds per condition (Figure 4b; Table 10 in Section C). Return on the 84 held-out clips of the first acquisition, standing up from a kneel, started at 0.817 and ended at 0.826–0.833 with replay and at 0.722–0.800 without it, and the mean loss of return of the five earlier types between their own step and the end fell from 0.029 to 0.001. Replay did not change the failed

rollouts on 822 clips of held-out types that no step trained, and it reduced the gain on the new type at two of the five steps.

Both sequences ran before the promotion gate was added. We ran a separate sequence of seven requests with the gate, from T_V, in a copy of the code, plans, and registry, so the project's registry still names T_V (Table 11 in Section C). Five requests trained the tracker. The gate promoted four candidates and rejected the fifth, trained for skipping: with it, the plan of the meditation skill acquired one request earlier completed in 0 of 32 worlds, against 28 of 32 with the current tracker, although meditation's training clips were in the replay. The averaged regression checks did not register the loss: on the held-out clips of all earlier acquisitions, meditation's among them, the candidate's return rose from 0.831 to 0.836. The check that failed ran the meditation skill's own plan. The factory withheld the skipping skill and kept the previous tracker. Meditation has one performer and four clips, two of them held out, so its share of the replay is small; whether this explains the loss is untested. The other two requests were served without training, one in 27 s by composing the newly acquired shadow boxing and push-up skills, and the seven requests cost $0.10 in API calls.

5.4 Reuse and composition

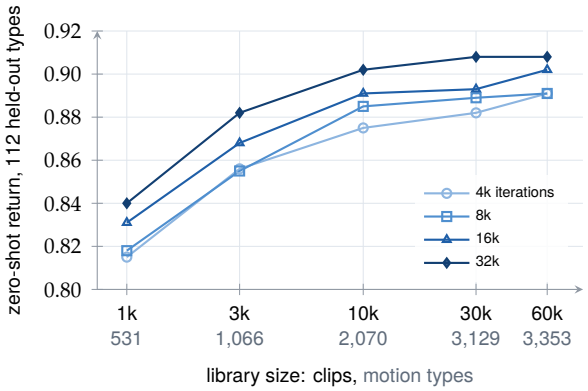
Registered skills compose at standing poses. With T_0, transitions between 16 registered skills completed in all 8,192 ordered-pair trials (256 pairs, 32 worlds each), with and without blending. The error that a transition adds in its first second has a median of 0.6–1.2 cm across the three settings of Table 12 and correlates with the compatibility gap between the two skills (Spearman 0.55–0.62); switching between agile skills in mid-motion is harder for general trackers [19] and was not tested. Handoffs between the tracker and the walking policy completed in 448 of 448 worlds, closed-loop `walk_to` reached its target in 224 of 224 worlds, and seven plans of 3–16 steps and up to 100 s completed in 896 of 896 worlds. In 60 planning sessions over 10 requests, the planner returned the right kind of answer, a plan or a refusal, in 59 and passed every check in 52, and its 53 submitted plans executed in 424 of 424 worlds. From one sentence, it wrote a nine-step plan of registered skills and closed-loop walking in 4 API calls, which T_K executed in 32 of 32 worlds, about 85 s each (Figure 10 in Section C).

6 Learning efficiency

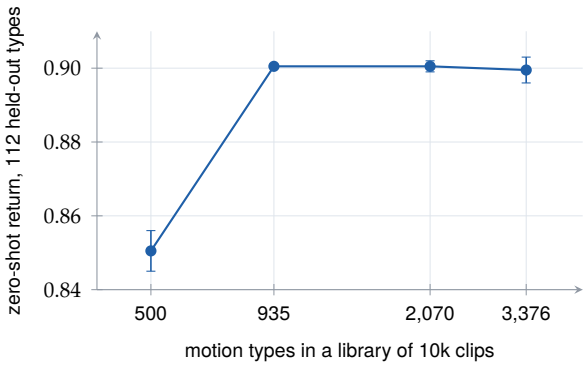
The training library can lower the cost of a new skill in two ways: the tracker may already perform the new motion, so that the factory registers it without training, or training may take less time. We report three measurements: the zero-shot generalization of trackers trained from scratch on libraries of controlled size and diversity (Section 6.1), the cost of acquisitions using the factory's procedure from two starting trackers (Section 6.2), and, as a supplementary analysis with a different training procedure, the fine-tuning iterations that new motion types need to reach fixed tracking thresholds. These measurements describe how cost changes as capability accumulates; the acquisition procedure itself stays fixed.

6.1 Zero-shot generalization

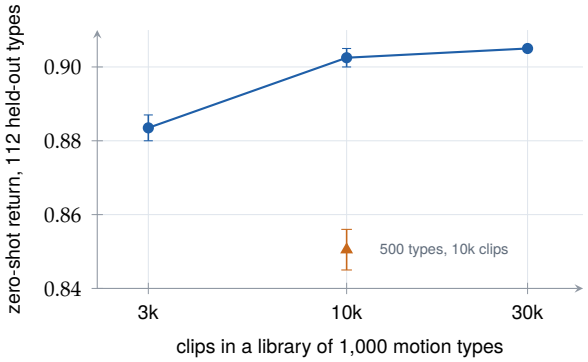
Five libraries of 1k, 3k, 10k, 30k, and 60k clips are drawn in one random order from the 13 training shards, so each is a subset of the next; they contain 531 to 3,353 motion types and 2.0 to 122.7 h of motion. These trackers are trained on BONES-SEED alone, each from scratch for 32k iterations on its own A800 GPU, and evaluated on the 112 held-out motion types after 4k, 8k, 16k, and 32k iterations (Figure 5a; Table 13 in Section C).



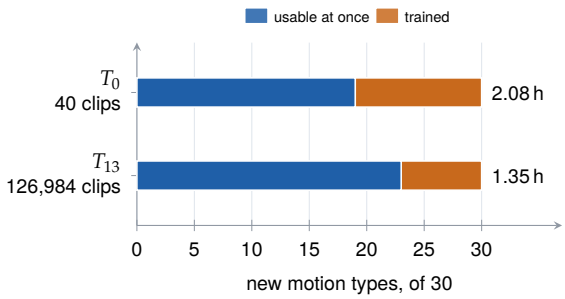
(a) Grid: every library trained from scratch.



(b) More motion types at a fixed number of clips.



(c) More clips of the same motion types.



(d) End-to-end acquisition of 30 new motion types.

Figure 5. Scaling and the cost of new skills. (a) Libraries of 1k–60k BONES-SEED clips, each a subset of the next, trained from scratch and evaluated after 4k–32k iterations (one seed): at every training budget, zero-shot return does not fall as the library grows; gray numbers give the motion types in each library. (b) Libraries of 10k clips with 500–3,376 motion types and (c) the same 1,000 types with 3k–30k clips, after 32k iterations, mean and range of two seeds; the triangle in (c) marks 500 types in 10k clips. (d) The 30 new motion types of Section 6.2 acquired with the factory's own procedure from T_0 and from T_{13}, before the promotion gate existed (one run): types usable at once and types that needed training, with the total wall-clock time of the acquisitions, which includes verification but not promotion; 27 types were registered from either start. Figure 11 in Section C shows the fits of the grid.

At each of the four budgets, zero-shot return did not fall as the library grew; after 32k iterations it rose from 0.840 at 1k clips to 0.908 at 30k and 60k clips. Eight times more training added 0.017–0.027 on every library, whereas going from 1k to 3k clips at the smallest budget added 0.041. From 30k to 60k clips, the number of clips doubled while the number of motion types grew by 7%, and return gained at most 0.009; a fit of the remaining error in the number of motion types predicts the 60k library from the smaller ones, and a fit in the number of clips overestimates it (Figure 11 in Section C).

Because types and clips grow together in the grid, two further tests varied them separately, with two seeds each (Figures 5b and 5c; Table 14 in Section C). With 10k clips, going from 500 to 935 motion types raised return on the held-out types from 0.851 to 0.900 and cut failed rollouts by more than 80%; with 2,070 and 3,376 types, the return stayed at 0.900. With the same 1,000 types, 3k, 10k, and 30k clips reached 0.884, 0.902, and 0.905. Where both were scarce, types mattered more: 1,000 types in 3k clips (0.884) exceeded 500 types in 10k clips (0.851). After 32k iterations, zero-shot

Table 2. Acquisition of 30 new motion types with the factory's procedure, from T_0 and from T_{13}, before the promotion gate was added; times include probing, training, verification, and registration, not promotion. Each registered skill was validated alone in ORCHESTRA in 32 randomized worlds; one skill from T_0 completed 31. Times are wall-clock times on one RTX 4090. The cost of training the starting trackers is excluded, so the comparison measures the marginal cost of acquisition. The three types not registered have no clip whose skill window fits the 12 s limit; T_0 trained one of them in vain. One run.

Start	Usable at once	Trained	Registered	Validated 32/32	Total	Per registered skill
T_0 (40 clips)	19	11	27	26	2.08 h	4.6 min
T_{13} (126,984 clips)	23	7	27	27	1.35 h	3.0 min

return thus levels off at about 0.90 once a library holds about a thousand motion types and ten thousand clips; longer training and larger models were not tested.

6.2 Acquisition cost

We acquired 30 new motion types, the 5 held-out types hardest for small libraries and 25 drawn at random, with the factory's procedure, once from T_0 (40 clips) and once from T_{13} (126,984 clips), before the promotion gate was added (Table 2; Figure 5d). The two trackers differ in their training data and in their training history, so the comparison does not isolate library size. From T_{13}, 23 of the 30 types passed the probe and 7 needed training, against 19 and 11 from T_0. Both registered 27 types, and the total wall-clock time fell from 2.08 to 1.35 h, from 4.6 to 3.0 min per registered skill. An acquisition with training took 624–655 s from either tracker and one without training 15–20 s, so the saving came from fewer training runs.

Fine-tuning to fixed thresholds. The factory trains for a fixed budget of 0.1 h. In a separate study without replay, we measured the fine-tuning iterations that the same 30 types need to reach fixed tracking thresholds, for seven trackers of the factory's history from T_0 to T_{13}, up to 2,000 iterations (Figure 12 and Table 15 in Section C). Return before fine-tuning rose from 0.833 to 0.914. The mean number of iterations after which a type reaches and keeps the probe criterion, counting 2,000 for a type that does not reach it, fell only from 653 to 473, almost all of it with the first shard: from T_1 on, 23 of the 30 types reached the criterion within 200 iterations and 6–7 not within 2,000. At a return of 0.75, the same mean fell from 460 to 60. A tracker trained on more data reached a return of 0.75 on a new type in fewer iterations, while the set of types that reached the probe criterion changed little.

Time of an acquisition. Table 3 divides the time of a request that trains into stages, before and after the gate was added. The factory spends about 7 min on probing and on training at the fixed budget of 0.1 h, and about 4 min on verification: evaluation on held-out performers and the regression sets, and validation of the new skill in ORCHESTRA. A request reaches its executed plan after 11–12 min with or without the gate. The gate runs after the plan; it took 394–408 s when the current tracker's results were cached and 698 s the first time, when it evaluated both trackers, so the promotion decision came about 18 min after the request, and 23 min after the first. In the four requests with cached results, verification and promotion together took 10–11 min, against about 7 min for training. The gate checks every registered plan and skill, and its number of checks grew from 51 to 55 over five requests.

Training cost with larger libraries. A larger library also slows training: on an A800, training throughput fell from 6.3 iterations/s at 1k clips to 0.9 iterations/s at 60k clips, because the tracker's training-time evaluation sweeps the whole library every 125 iterations, and the 60k library needed 9.8 h to

Table 3. Time of a request that trains the tracker, by stage, in wall-clock seconds on one RTX 4090, from recorded wall-clock timings. *Before the gate*: medians of the 12 acquisitions with training in the factory's records, and the range over requests with one training (Table 16). *With the gate*: the five requests that trained in the gated sequence (Table 11). The gate runs after the plan has executed; its first run evaluated both trackers. One run each.

Stage		Before the gate	With the gate
Acquisition	probe	9	7–11
	training, fixed budget of 0.1 h	410	400–407
Verification	evaluation on held-out performers and regression sets	238	215–228
	registration and validation of the new skill	0 + 9	9–12
Request to executed plan		671–709	664–695
Promotion gate		–	394–408; first 698
Request to promotion decision		–	1,058–1,093; first 1,393

match what the 30k library reached in 5.5 h. Moving that evaluation to five additional GPUs, with a small patch to InstantTrack, raised the throughput of the 60k library to 2.99 iterations/s; a library of 100k clips would need about 93 GiB to load and does not fit in 80 GB (Section C.8).

7 Extensions

7.1 Interaction experts

The tracker observes neither a box nor its load. Given a box-carrying reference from OmniRetarget's G1 data, with a 1 kg simulated box placed where the reference expects it, the tracker completed the body motion in 32 of 32 worlds and left the box 0.67–0.68 m from its target (Table 22 in Section C); other general trackers fail in the same way on object tasks [22, 60]. The factory therefore trains interaction policies of two predefined families on mjlab's tracking task: box-carrying policies, which observe the box's pose and target, and climbing policies, which train with the clip's blocks as static terrain.

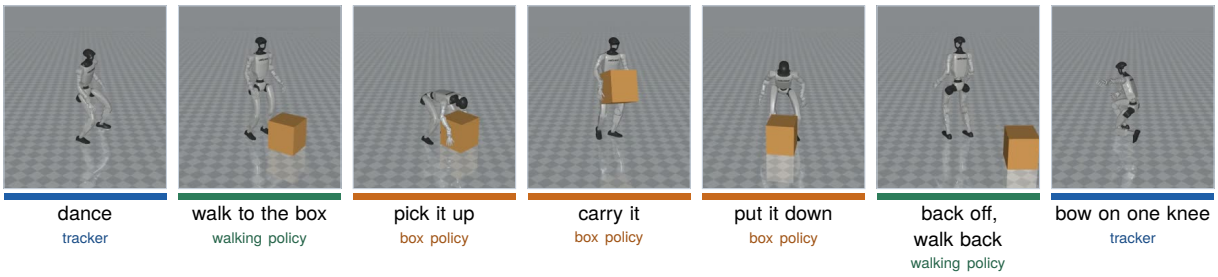


Figure 6. Three kinds of experts in one continuous run. The tracker dances, the walking policy walks to a box 3 m ahead, a box policy picks it up, carries it about 1.4 m and puts it down, the walking policy backs off and walks back, and the tracker bows on one knee. The bar under each frame marks the expert in control. All 32 randomized worlds completed the plan, with the box put down 6.7 cm from its target on average, 0 resets, and 0 robot or object state writes. Frames show world 0.

An interaction policy has to be trained on the state in which it receives the robot. A box policy trained on its clip succeeded in 29 of 32 worlds when evaluated alone and in 0 of 32 when ORCHES-

TRA's walking policy handed it the robot, standing still in the walking policy's own stance. After fine-tuning on episodes that partly start from that stance, walk–carry–return plans completed in 32 of 32 worlds, also inside longer plans with tracker skills before and after (Figure 6). Four further box clips turn the robot by 113–210° while carrying the box; their policies succeeded in 32, 26, 26, and 30 of 32 worlds, and because their checkpoints vary strongly, we selected them by their results in ORCHESTRA. Five climbing policies trained in the same way step onto, climb, or vault obstacles of 0.40–0.76 m, and each completed its plan in 32 of 32 worlds under each of two randomization seeds.

Acquisition by the loop. For a 0.70 m wall that no registered skill was made for, the request was "Get over the wall in front of you and stand on the other side." The planner searched the library once and requested a clip that crosses the wall sideways. The factory trained a new climbing policy in stages and ran the plan in 32 worlds after each stage. All 32 worlds completed the plan after both stages, and by the skill's own end test the first stage succeeded in 7 of 32 worlds and the second in 30 of 32. The loop stops training once a skill succeeds in at least 90% of the worlds, which it reached 76 min after the request, at an API cost of $0.013. The same procedure failed on a 0.65 m platform with a jump down: the policy never learned the second half of the clip, and the skill was not registered. In both cases the factory created an instance of a predefined family; creating new expert families automatically is a direction for future work (Section 8). Acquiring an interaction skill leaves the shared tracker's weights unchanged.

7.2 Skills from video

The factory accepts motions estimated from web videos once GEM-X [37] and soma-retargeter [38] have turned them into G1 motions. For a 14 s dance with dribbling, turns, and kicks, B_5 tracked the motion without a fall but below the probe criterion (return 0.774), so the factory trained for 0.1 h, which produced T_V; the return did not rise (0.766 afterwards), and the regression sets stayed level. The retargeted motion keeps jitter and foot sliding and drives the waist past the G1's joint limit for 118 frames, which is a plausible cause. Five Creative Commons videos, tai chi, bodyweight squats, jumping jacks, a karate kata, and a line dance, were then processed one after another with a probe level of 0.70 for estimated motions. T_V performed all five without training (probe returns 0.745–0.935), and every resulting skill completed in 32 of 32 worlds with a mean body position error of 3.3–6.9 cm (Figure 13 and Table 17 in Section C). Each video took 6–8 min from its URL to a validated skill, about 30 s of it in the factory. Compared with the retargeted reference, the simulated feet stay closer together in wide stances, 0.43 instead of 0.58 m in the tai chi form and 0.46 instead of 0.60 m in the kata, and the jumping jacks lose most of their leg spread; the return did not drop enough for the probe to trigger training.

7.3 Other physics engines and a second robot

The factory exports a tracker as a deployment package: the policy in ONNX, its configuration, the MuJoCo scene of training, and the reference of every skill; the policy reads only joint encoders and the IMU. A physics backend implements three functions, setting the initial state, advancing one control step, and reading the state, which we implemented for classic MuJoCo, Genesis [7], and Isaac Sim through Isaac Lab [39]. These tests concern the exported tracker; the loop itself runs in the training simulator. On 108 clips of held-out motion types, B_5 scored within 0.006 of the training simulator in the three other engines, and every difference between engines was smaller than the effect of one 20 ms step of action delay, which lowered LAFAN1 return from 0.866 to 0.788 (Table 18 in Section C). We used the same pipeline to train a tracker for the Unitree H1-2 on 40 LAFAN1

clips retargeted to it [33], without changes to the upstream training code but with assumed motor armature values; after 0.3 h it reached a return of 0.744 with 2 of 40 clips failing, against 0.82 and no failures after 3–5 min for the G1.

8 Discussion

Evidence for skill-level RSI. The experiments evaluate skill-level RSI through retention, cross-request reuse, and regression checks. Replaying earlier acquisitions reduced mean forgetting from 0.029 to 0.001 in the controlled five-step study. Acquired skills served later requests through composition; D7 took 27 s when its component skills were reused, compared with 688 s when they were acquired again. In a separate seven-request sequence, the gate rejected a candidate that reduced completion of the meditation skill's plan from 28/32 to 0/32 worlds, although the aggregate regression checks passed. These results correspond to level 1 in [11]: capability gains persist and benefit subsequent tasks. Level 2 requires persistent, system-generated revisions to the improvement procedure. The current factory uses fixed acquisition, replay, and promotion rules.

Open questions. Three questions follow from the measurements. First, the probe criterion decides when the factory trains, and of the 15 requests with training whose plans we reran with the tracker from before the training (Section 5.2), 12 completed equally well without it. A rule that runs the request's plan with the current tracker first and trains only when the plan fails or misses an error target would skip the most expensive stage in those cases. The planner should also request the motion type that a request names instead of approximating it with registered skills, the most common benchmark failure, and check requirements that motion descriptions omit, such as an arm posture. Second, verification and promotion grow with the registry. Reusing the current tracker's results cut the gate from 698 to about 400 s, and prioritizing checks for skills likely to be affected by an update could reduce the cost further, but the rejection in Section 5.3 came from one skill's plan while the averaged regression sets passed, so per-skill checks have to remain. Third, execution failures do not yet start an acquisition. Every monitor stop names the skill, the time, and the rule that fired, which is the input a diagnosis needs; the current thresholds have stopped only failures that ended in a fall, and a stop takes no safety action. The factory's interface to a policy is a set of records, a training job, a policy artifact, an evaluation result, and a skill manifest, so other policy families, such as diffusion policies or vision-language-action models, can be added as backends that train and evaluate in their own way.

Evaluation scope. The evaluation covers simulated humanoid control, with the full loop on one robot. Most system-level experiments, including the request sequences and the benchmark, ran once (Section B), and plan completion and benchmark acceptance check tracking and the components of a request, not whether the executed motion fulfils the request's intent.

9 Conclusion

Robot Skill Factory connects skill acquisition for a shared humanoid tracker to verification, tracker versions, and continuous execution. On a simulated G1 and one GPU, it met the acceptance criteria of 30–31 of 37 benchmark requests, against 21 without acquisition, kept earlier acquisitions when it replayed their training clips, and rejected an update that reduced completion of a previously acquired skill's plan. The cost of a new skill depends mainly on how often the probe triggers training, and verification and promotion take longer than the training they check. The experiments provide evidence for skill-level RSI in simulated humanoid control through persistent acquisition, retention, and reuse across requests.

References

[1] Michael Ahn, Anthony Brohan, et al. Do as I can, not as I say: Grounding language in robotic affordances. In *Conference on Robot Learning* (*CoRL*), 2022.

[2] Donghoon Baek, Sang-Hun Kim, and Sehoon Ha. BAT: Balancing agility and stability via online policy switching for long-horizon whole-body humanoid control. *arXiv preprint arXiv:2604.01064*, 2026.

[3] Bones Studio. BONES-SEED. Hugging Face dataset, https://huggingface.co/datasets/bones-studio/seed. Academic non-commercial research use; accessed November 2026.

[4] Fuyi Chen. BlinkMimic: Whole-body motion tracking for the Unitree G1 in minutes on one GPU. https://github.com/fuyichen0725/blinkmimic, 2026.

[5] Sirui Chen, Shibo Zhao, Zhen Wu, Jiaman Li, Guanya Shi, and C. Karen Liu. SceneBot: Contact-prompted general humanoid whole body tracking with scene-interaction. *arXiv preprint arXiv:2606.27581*, 2026.

[6] Ikjun Choi, Geonho Leem, and Jungdam Won. InstantMimic: A high performance system for learning physics-based skills in seconds. *arXiv preprint arXiv:2609.09821*, 2026.

[7] Genesis Authors. Genesis: A generative and universal physics engine for robotics and beyond. https://github.com/Genesis-Embodied-AI/genesis-world, 2024.

[8] Google DeepMind and NVIDIA. MuJoCo Warp: GPU-accelerated MuJoCo with NVIDIA Warp. https://github.com/google-deepmind/mujoco_warp, 2025.

[9] Zhongkai Gu, Yang Liu, Wei Zhu, and Mitsuhiro Hayashibe. ReContact: Reconstruct and retarget human contacts from video for contact-rich humanoid manipulation. *arXiv preprint arXiv:2611.02968*, 2026.

[10] Yu Guan, Zekun Qi, Chenghuai Lin, Xuchuan Chen, Dairu Liu, et al. LIMMT: Less is more for motion tracking. *arXiv preprint arXiv:2606.06953*, 2026.

[11] Xiaofeng Han, Kaize Luo, Zenghuang Fu, Zichong Zhu, Zhe Feng, et al. Recursive self-improvement in physical AI: Mechanisms, evidence, and evaluation. *arXiv preprint arXiv:2610.38266*, 2026.

[12] Félix G. Harvey, Mike Yurick, Derek Nowrouzezahrai, and Christopher Pal. Robust motion in-betweening. *ACM Transactions on Graphics* (*Proceedings of ACM SIGGRAPH*), 39(4):60:1–60:12, 2020.

[13] Zhewen He, Hao Huang, Geeta Chandra Raju Bethala, Chong Yu, Tao Chen, Anthony Tzes, and Yi Fang. Continual humanoid motion learning. *arXiv preprint arXiv:2610.04231*, 2026.

[14] Gio Huh et al. HomeBody: A humanoid that explores, remembers, and acts on its own. https://tml.stanford.edu/homebody/, 2026. Project page and code.

[15] Huy-Quang-Dao. InstantTrack: Whole-body motion tracking for the Unitree G1 in minutes on one GPU. https://github.com/Huy-Quang-Dao/instantTrack, 2026. Version 1.3.0.

[16] Mazeyu Ji, Xuanbin Peng, Fangchen Liu, Jialong Li, Ge Yang, Xuxin Cheng, and Xiaolong Wang. ExBody2: Advanced expressive humanoid whole-body control. *arXiv preprint arXiv:2412.13196*, 2024.

[17] Minku Kim, Kuan-Chia Chen, Aayam Shrestha, Li Fuxin, Stefan Lee, and Alan Fern. Humanoid Hanoi: Investigating shared whole-body control for skill-based box rearrangement. *arXiv preprint arXiv:2602.13850*, 2026.

[18] Haoran Lang, Haotao Lu, Shiyu Sang, Haoyang Luo, Guo Chen, et al. EmbodiRSI: Recursive self-improvement for data-efficient robot adaptation. *arXiv preprint arXiv:2609.38905*, 2026.

[19] Yuen-Fui Lau, Qihan Zhao, Yinhuai Wang, Runyi Yu, Hok Wai Tsui, Qifeng Chen, and Ping Tan. Switch: Learning agile skills switching for humanoid robots. *arXiv preprint arXiv:2604.14834*, 2026.

[20] Dongting Li, Xingyu Chen, Qianyang Wu, Bo Chen, Sikai Wu, et al. HAIC: Humanoid agile object interaction control via dynamics-aware world model. *arXiv preprint arXiv:2602.11758*, 2026.

[21] Jiaman Li, Jiajun Wu, and C. Karen Liu. Object motion guided human motion synthesis. *arXiv preprint arXiv:2309.16237*, 2023.

[22] Xinyao Li, Xialin He, Runpei Dong, and Saurabh Gupta. ContactMimic: Humanoid object interaction via contact control. *arXiv preprint arXiv:2607.08742*, 2026.

[23] Xuebing Li et al. Agentic robot creation through generation, verification, and revision. *arXiv preprint arXiv:2610.17881*, 2026.

[24] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.

[25] Qiayuan Liao, Takara E. Truong, Xiaoyu Huang, Guy Tevet, Koushil Sreenath, and C. Karen Liu. BeyondMimic: From motion tracking to versatile humanoid control via guided diffusion. *arXiv preprint arXiv:2508.08241*, 2025.

[26] Qiayuan Liao, Yiyang Shao, Junfeng Long, Kevin Zakka, Roberto Horowitz, Pieter Abbeel, and Koushil Sreenath. SuperMimic: Scaling humanoid whole-body control with one differentiable world model. *arXiv preprint arXiv:2610.28480*, 2026.

[27] Yutang Lin, Jieming Cui, Yixuan Li, Baoxiong Jia, Yixin Zhu, and Siyuan Huang. LessMimic: Long-horizon humanoid interaction with unified distance field representations. *arXiv preprint arXiv:2602.21723*, 2026.

[28] Zhuo Lin, Sirui Xu, et al. InterEvolve: Test-time evolution of reward programs for humanoid loco-manipulation. *arXiv preprint arXiv:2610.02196*, 2026.

[29] Qingzhou Lu, Chenxin Liu, Xinming Wang, Liu Cao, Yanjiang Guo, Mengdi Xu, and Jianyu Chen. HITCH: Teaching a vision-language model to operate a humanoid through a whole-body harness. *arXiv preprint arXiv:2611.08039*, 2026.

[30] Runyu Lu et al. ASPIRE: Agentic /Skills discovery for robotics. *arXiv preprint arXiv:2607.00272*, 2026.

[31] Zhengyi Luo, Jinkun Cao, Josh Merel, Alexander Winkler, Jing Huang, Kris Kitani, and Weipeng Xu. Universal humanoid motion representations for physics-based control. *arXiv preprint arXiv:2310.04582*, 2023.

[32] Zhengyi Luo et al. SONIC: Supersizing motion tracking for natural humanoid whole-body control. *Science Robotics*, 2026. arXiv:2511.07820.

[33] lvhaidong. LAFAN1 retargeting dataset. Hugging Face dataset, https://huggingface.co/datasets/lvhaidong/LAFAN1_Retargeting_Dataset. Unitree G1 and H1-2 retargets of LAFAN1; CC BY-NC-ND 4.0.

[34] Yubiao Ma, Han Yu, Kai Guo, Changtai Lv, Zhengquan Mao, Boyang Xing, Xuemei Ren, and Dongdong Zheng. Extreme-RGMT: Continual learning of highly dynamic skills for robust generalist humanoid control. *arXiv preprint arXiv:2607.20110*, 2026.

[35] Motphys. Uni-SuperMimic. https://github.com/Motphys/Uni-SuperMimic, 2026.

[36] Loc X. Nguyen, Avi Deb Raha, Huy Q. Le, Eui-Nam Huh, Dusit Niyato, and Choong Seon Hong. Self-evolving AI for humanoids: Mechanisms, safety, and evaluation of post-deployment self-improvement. *arXiv preprint arXiv:2609.13236*, 2026.

[37] NVIDIA. GEM-X: Full-body and hand pose estimation from image and video. https://github.com/NVlabs/GEM-X, 2026. Commit 4aef145.

[38] NVIDIA. SOMA retargeter. https://github.com/NVIDIA/soma-retargeter, 2026. Version 0.2.1.

[39] NVIDIA, Mayank Mittal, Pascal Roth, James Tigue, Antoine Richard, et al. Isaac Lab: A GPU-accelerated simulation framework for multi-modal robot learning. *arXiv preprint arXiv:2511.04831*, 2025.

[40] Xue Bin Peng, Pieter Abbeel, Sergey Levine, and Michiel van de Panne. DeepMimic: Example-guided deep reinforcement learning of physics-based character skills. *ACM Transactions on Graphics*, 37(4), 2018.

[41] Zekun Qi, Xuchuan Chen, Dairu Liu, Chenghuai Lin, Yunrui Lian, et al. Humanoid-GPT: Scaling data and structure for zero-shot motion tracking. *arXiv preprint arXiv:2606.03985*, 2026.

[42] Ziyi Sun et al. CHOREO: Every humanoid skill as a trajectory. *arXiv preprint arXiv:2609.22274*, 2026.

[43] Chen Tessler, Yifeng Jiang, et al. ProtoMotions3: An open-source framework for humanoid simulation and control. https://github.com/NVlabs/ProtoMotions, 2025.

[44] Chen Tessler, Yifeng Jiang, et al. METATRON: A promptable whole-body execution layer for humanoids. *arXiv preprint arXiv:2610.38027*, 2026.

[45] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012.

[46] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

[47] Maggie Wang, Lars Osterberg, Stephen Tian, Ola Shorinwa, Joseph Bowkett, Daniel Pastor, Jiajun Wu, and Mac Schwager. InSight: Self-guided skill acquisition via steerable VLAs. *arXiv preprint arXiv:2606.24884*, 2026.

[48] Peidong Wang, Ying Chang, Zhiming Ma, Xiaocui Yang, Shi Feng, et al. AdaRSI: On-demand recursive self-improvement for embodied agents. *arXiv preprint arXiv:2610.20474*, 2026.

[49] Yufei Wang, Zhou Xian, Feng Chen, Tsun-Hsuan Wang, Yian Wang, Katerina Fragkiadaki, Zackory Erickson, David Held, and Chuang Gan. RoboGen: Towards unleashing infinite data for automated robot learning via generative simulation. In *International Conference on Machine Learning (ICML)*, 2024.

[50] Haoyang Weng, Yitang Li, Nikhil Sobanbabu, Zihan Wang, Zhengyi Luo, Tairan He, Deva Ramanan, and Guanya Shi. HDMI: Learning interactive humanoid whole-body control from human videos. *arXiv preprint arXiv:2509.16757*, 2025.

[51] Tianyi Xie, Haotian Zhang, Jinhyung Park, Zi Wang, Bowen Wen, et al. GRAIL: Generating humanoid loco-manipulation from world models. *arXiv preprint arXiv:2606.05160*, 2026.

[52] Lujie Yang, Xiaoyu Huang, Zhen Wu, et al. OmniRetarget: Interaction-preserving data generation for humanoid whole-body loco-manipulation and scene interaction. *arXiv preprint arXiv:2509.26633*, 2025.

[53] Runyi Yu, Xiaoyi Lin, Ji Ma, Yinhuai Wang, Koukou Luo, et al. OmniContact: Chaining meta-skills via contact flow for generalizable humanoid loco-manipulation. *arXiv preprint arXiv:2606.26201*, 2026.

[54] Xichen Yuan, Zhe Li, et al. RoboForge: Physically optimized text-guided whole-body locomotion for humanoids. *arXiv preprint arXiv:2603.17927*, 2026.

[55] Renos Zabounidis et al. SCALAR: Learning and composing skills through LLM guided symbolic planning and deep RL grounding. *arXiv preprint arXiv:2603.09036*, 2026.

[56] Kevin Zakka, Qiayuan Liao, Brent Yi, Louis Le Lay, Koushil Sreenath, and Pieter Abbeel. mjlab: A lightweight framework for GPU-accelerated robot learning, 2026.

[57] Weishuai Zeng, Kangning Yin, Xiaojie Niu, Shunlin Lu, Weixiang Zhong, et al. Scaling behavior foundation model for humanoid robots. *arXiv preprint arXiv:2607.15163*, 2026.

[58] Yuanhang Zhang, Yifu Yuan, Prajwal Gurunath, Ishita Gupta, Shayegan Omidshafiei, Aliakbar Agha-mohammadi, Marcell Vazquez-Chanlatte, Liam Pedersen, Tairan He, and Guanya Shi. FALCON: Learning force-adaptive humanoid loco-manipulation. *arXiv preprint arXiv:2505.06776*, 2025.

[59] Yucheng Zhang, Sirui Xu, Jinhong Li, Liuyu Bian, Anatulya Nandi, et al. InterMimicGen: Scaling humanoid loco-manipulation through self-evolving motion imitation. *arXiv preprint arXiv:2610.06850*, 2026.

[60] Siheng Zhao, Yanjie Ze, et al. ResMimic: From general motion tracking to humanoid whole-body loco-manipulation via residual learning. *arXiv preprint arXiv:2510.05070*, 2025.

A Implementation details

Tracker architecture. Figure 7 shows the policy. A motion encoder maps the next 10 reference frames (730 values: root position and orientation relative to the current reference frame, root velocities, joint positions and velocities) to a 32-dimensional latent code. A decoder maps the code and the last 10 proprioceptive observations to 29 joint position targets. Each observation has 99 values: joint positions and velocities, root angular velocity, the gravity direction in the body frame, the previous action, and the reference root orientation relative to the robot's. A skip connection adds the reference joint positions to the output, so the network learns a correction to them. Implicit PD actuators follow BeyondMimic's actuator model [25]. The policy has 9.12M parameters (encoder 4.26M, decoder 4.86M); training also uses the world model (2.31M) and a critic (0.27M), and evaluation uses an exponential moving average of the policy weights.

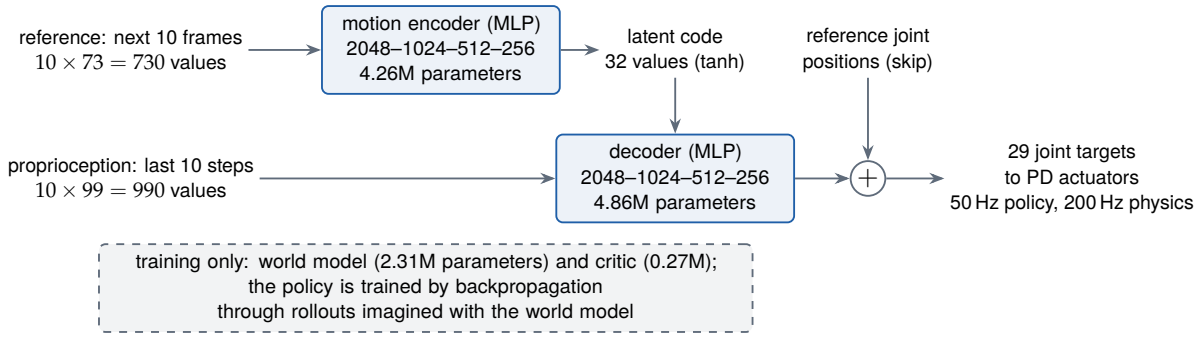


Figure 7. The tracking expert. The encoder sees only the reference, so its output can be computed offline for each skill; the decoder combines the latent code with ten steps of proprioception (joint positions and velocities, root angular velocity, gravity direction, previous action, and reference root orientation relative to the robot) and adds a correction to the reference joint positions. The deployed policy has 9.12M parameters.

Tracker interface. Table 4 lists the inputs and output of the tracking expert. The policy runs at 50 Hz and the physics at 200 Hz. Actuators follow BeyondMimic's model [25]: implicit PD control with gains set for a 10 Hz natural frequency and a damping ratio of 2 on each joint's armature, and an action scale of $0.25\,\tau_{\max}/k_p$ per joint.

Table 4. Inputs and output of the tracking expert. During training, uniform noise is added to the observation: ± 0.01 rad on joint positions, ± 0.5 rad/s on joint velocities, ± 0.2 rad/s on angular velocity, ± 0.05 on the gravity direction and on the reference orientation.

Signal	Size	Content
Observation (per step, last 10 steps)	99	joint positions minus the default pose (29), joint velocities (29), root angular velocity in the body frame (3), gravity direction in the body frame (3), previous action (29), reference root orientation relative to the robot, 6D (6)
Reference (per frame, next 10 frames)	73	root position change and orientation (6D) relative to the current reference frame (9), root linear and angular velocity in the root frame (6), joint positions (29), joint velocities scaled by 0.1 (29)
Output	29	joint position targets: default pose plus action scale times action, tracked by implicit PD at 200 Hz

Interaction expert. The box policies use mjlab's tracking actor with object terms, 178 values per step and no history: the reference joint positions and velocities at the current frame (58), the reference torso position and orientation in the robot's torso frame (9), base linear and angular velocity (6), joint positions and velocities (58), the previous action (29), the box's position and orientation in the torso frame (9), and the box's target pose, the reference box pose moved with the robot, in the torso frame (9). Actions use the same G1 action space as the tracker, so ORCHESTRA passes them through the same simulation step. The climbing policies use the same actor without the box terms, 160 values per step; the terrain is static geometry in the simulated scene, which the policy does not observe.

Planner and loop. `check_plan` verifies skill names and step parameters, checks entry and exit compatibility, and predicts where the robot will be after each step with a walking model calibrated on measured executions; for example, backward walking reaches 0.90 and sideways walking 0.84 of the commanded speed, with a 0.3 s lag. For a plan with interaction steps it also predicts where each object ends up, rejects a plan whose object placement departs from the scene by more than 5 cm or by a yaw that is not a multiple of 90°, and accepts a terrain skill only on the terrain it was made for. Algorithm 1 lists the control flow for one request. A requested interaction clip takes the interaction branch of the factory instead of lines 6–11: it trains in stages and runs the request's plan again after each stage, and it trains another stage only while the new skill succeeds in fewer than 90% of the worlds. Every planning round sees the registry as the previous round left it, so skills registered for one request serve later requests through composition alone.

Algorithm 1 Serving one request with skill acquisition and automatic promotion (motion branch).

Input: request q and scene; registry $\mathcal{R}$ and its current tracker π_0; motion library $\mathcal{M}$; $N = 32$ worlds; $\pi \leftarrow \pi_0$.

1: **for** planning round $r = 1, \ldots, 4$ **do**
2: the planner reads $\mathcal{R}$, drafts a plan P, and validates it with `check_plan`
3: **if** P is a plan **then** execute P with π in N randomized worlds; **break**
4: **if** the planner refuses q, or $r = 4$, **then** record the refusal and its reason; **break**
5: the planner searches $\mathcal{M}$ and requests a motion type c ▷ a skill is missing
6: probe π on up to 40 clips of c, 2 rollouts each
7: **if** no rollout fails **and** the mean return is at least 0.85 **then** register one skill of c with π (Section 3.3)
8: **else**
9: $\pi' \leftarrow$ warm-start π on the training performers of c, replay, and earlier acquisitions (0.1 h)
10: evaluate π' on held-out performers, earlier motions, held-out types, and LAFAN1
11: register one skill of c with π', chosen by the clip criteria of Section 3.3; $\pi \leftarrow \pi'$
12: **if** $\pi \neq \pi_0$ **then** run the regression gate on π: promote it, or keep π_0 and withdraw every skill registered with a tracker version trained since π_0, including intermediate versions of this request
13: **return** the execution report or the refusal, and the promotion decision

Promotion gate. The gate compares a candidate tracker with the current one on the same worlds and clips and promotes the candidate only if every check passes. *Plans*: every registered plan and every registered motion skill alone, each in 32 randomized worlds in ORCHESTRA; each plan may complete at most one world fewer than with the current tracker, all plans together at most two fewer, and no run may reset or write state. *Regression sets*: the first shard's evaluation sample, the held-out motion types, LAFAN1, and the held-out performers of every earlier acquisition along the candidate's chain of warm starts; on each, the mean return may drop by at most 0.02 and the share of failed rollouts may rise by at most one percentage point. *New motion*: the held-out performers of the acquisition that trained the candidate, under the same rule. A check whose data is missing

rejects the candidate unless it is explicitly allowed to be skipped, which is recorded. The current tracker's results are kept per checkpoint and reused for unchanged plans and sets, so after the first decision the gate mostly evaluates the candidate. Every decision is appended to the registry's promotion log with the candidate's checksum, lineage, training set, and acquisition record, every check with the values of both trackers, and the gate's timing; Table 11 summarizes the log of the gated sequence.

Compute. Table 5 lists the cost of each stage. All tracker and interaction training runs 4,096 simulated worlds.

Table 5. Compute per stage. Peak memory is that of the whole GPU.

Stage	GPU	Wall-clock time	Peak memory
Tracker training, 40 LAFAN1 clips (usable, 0 failures)	RTX 4090	4.5–5.0 min	16.6–22.5 GB
Sharded training, 13 BONES-SEED shards	RTX 4090	88 min	22.8–23.5 GiB
Warm-start acquisition of one motion type	RTX 4090	6.6 min, 1,902 iterations	19.2 GiB
Probe and direct registration of one motion type	RTX 4090	about 20 s	–
Promotion gate, one candidate: current tracker cached; first run	RTX 4090	394–408 s; 698 s	–
Scaling grid, one library, 32k iterations	A800	1.4–9.8 h	up to 62 GB
Walking policy, 6,000 iterations	RTX 4090	50 min	3.1 GB
Box policy, 6,000 iterations	RTX 4090	87 min	3.6 GB
Hand-off fine-tuning of the box policy, 3,000 iterations	RTX 4090	53 min	14.7 GB
New box skill from a library clip, trained and verified	RTX 4090	55 min	14.9 GB
Climbing policy, 6,000 + 3,000 iterations	A800	1.85–2.17 h + 1.00–1.17 h	–
Video to G1 motion, one 12–14 s clip	RTX 4090	1.8–3.1 min	–
H1-2 tracker, 40 LAFAN1 clips	RTX 4090	0.3 h	16.8 GB
Execution of the nine-step performance, 32 worlds × 85 s	RTX 4090	43 s	1.0 GB

B Experimental setup

Data and splits. LAFAN1 is the 40-clip G1 retarget [33] of the LAFAN1 motion capture set [12]; its clips are in the training data of every tracker the factory produces, while the trackers of the scaling grid are trained on BONES-SEED alone. BONES-SEED [3] contributes 129,485 feasible motions. Of these, 112 motion types (2,541 clips) are held out from all training, except where an acquisition trains on one of them, and the remaining 126,944 clips form 13 training shards, assigned as whole takes, round-robin within each category. When the factory trains on a motion type, about 20% of that type's performers are held out for verification. Video motions come from one YouTube dance video and five YouTube videos under a Creative Commons Attribution license, and robot–object and robot–terrain clips from OmniRetarget's G1 data [52].

Evaluation. Tracker evaluations run InstantTrack's evaluation without domain randomization, from each clip's first frame, with 2 rollouts per BONES-SEED clip and 4 per LAFAN1 or video clip. The per-step reward, with control step $\Delta t = 0.02\,\mathrm{s}$, is

$$r_t = \Delta t \left[e^{-\bar{e}_p/0.3^2} + e^{-\bar{e}_o/0.4^2} + e^{-\bar{e}_v/1.0^2} + e^{-\bar{e}_\omega/3.14^2} + \tfrac{1}{2} e^{-e_{rp}/0.3^2} + \tfrac{1}{2} e^{-e_{ro}/0.4^2} - 0.1 \left\| a_t - a_{t-1} \right\|^2 - 2\,\ell_t \right],$$

where $\bar{e}_p$, $\bar{e}_o$, $\bar{e}_v$, and $\bar{e}_\omega$ are the squared errors of position (m), orientation angle (rad), linear velocity, and angular velocity, averaged over the 14 tracked bodies of a reference moved to the robot's horizontal position and heading; e_{rp} and e_{ro} are the squared root position and orientation errors; and ℓ_t is the summed violation of joint limits set at 90% of the range. A rollout's return is $\sum_t r_t / (5\,\Delta t\,(T-1))$ for a clip of T frames. A rollout fails when $|z - z^{\mathrm{ref}}| > 0.25\,\mathrm{m}$ for the root height or $|R_{zz} - R_{zz}^{\mathrm{ref}}| > 0.8$ for the vertical component of the root's vertical axis; a failed rollout stops scoring, and its remaining steps count as zero. These definitions are those of InstantTrack v1.3 (commit `9474e74`, `src/instanttrack/mdp.py` and `evaluate.py`), unmodified. The reported return is the mean over all rollouts of all clips, each clip weighted equally. Plan executions in ORCHESTRA use 32 worlds with the tracker's training-time domain randomization: friction, center of mass, joint offsets, and observation noise and delay.

Sharded training. The tracker keeps about 1.36 kB per motion frame on the GPU, so the 28.45M frames of the filtered library need 38.6 GB and do not fit on a 24 GB card. Each shard holds about 9,770 clips (2.9 GB) and replays the 40 LAFAN1 clips three times.

Compute and seeds. The loop, the tracker training of Section 4, and every plan execution run on one RTX 4090 with 24 GB of memory. The scaling grid runs on A800 GPUs with 80 GB, one library per GPU with no other process on it; the five climbing policies of Section 7.1 were trained on A800 GPUs, and the box policies on either machine. Tracker training on LAFAN1, the controlled acquisition study, and the two diversity tests use two seeds, and the controlled replay study three; every other training result uses one seed. The diversity tests ran on A800 GPUs and the controlled replay study on the RTX 4090.

Tracker lineage. Figure 8 draws the lineage of the trackers; every parent comes from the warm-start record of the training run, and a warm start continues the parent's full training state. Chain A, an earlier sequence of five requests from T_K without replay of earlier acquisitions, registered 8 skills that were moved out of the registry when chain B ran.

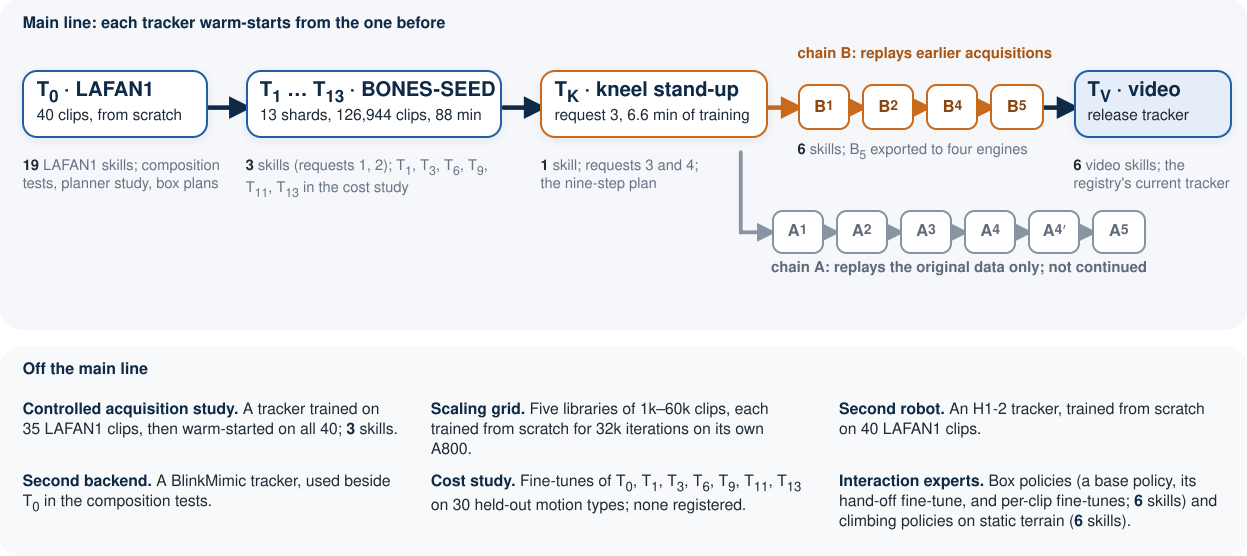


Figure 8. Exact tracker lineage, from the warm-start record of every training run. Each tracker on the main line warm-starts from the one before it. T_0 is trained on LAFAN1; $T_1, \ldots, T_{13}$ add one BONES-SEED shard each; T_K acquires standing up from a kneel; two sequences of five requests start from T_K; chain B, which replays earlier acquisitions, continues to T_V, which acquired a dance from video and is the current tracker of the project's registry, and chain A, an earlier sequence without that replay, was not continued. The note under each tracker gives the number of registered skills it verified and the experiments that used it. The lower panel shows trackers trained beside the main line, for controlled studies, a second backend, a second robot, and object interaction.

C Additional results

C.1 Sharded training on BONES-SEED

T_0 first reached a return of 0.82 after 2.7–3.1 min in both seeds. From T_0 to T_{13}, return on the evaluation samples of the trained shards rises from 0.874–0.889 to 0.935–0.943, with 5 failed rollouts in 7,892; the first shard's sample rises from 0.928 after its own shard to 0.943 after all 13, and LAFAN1 stays at 0.852 (0.858 before, 0 failures in 160 rollouts). The 13 shards peaked at 22.8–23.5 GiB of GPU memory. A tracker trained on the whole library at once with twice our GPU memory [35] scores 0.012–0.013 higher on the three shards we compared but fails 36.9% of LAFAN1 rollouts. Table 6 compares the trackers of Section 4, and Figure 9 shows return on three evaluation sets after each shard of sequential training.

Table 6. Trackers on BONES-SEED and LAFAN1, as normalized return (higher is better) / share of failed rollouts (lower is better). Trained shards are the evaluation samples of the 13 training shards. T_0 is the starting point of sharded training and T_{13} its end. USM's tracker was trained on the whole library, so the held-out types are in its training data.

Tracker	Trained shards	Held-out types	LAFAN1	Training
T_0 (LAFAN1 only)	0.874–0.889 / 0.8–1.6%*	0.827 / 2.9%	0.858 / 0.6%	–
T_{13}, 13 shards	0.935–0.943 / 0–0.3%	**0.901** / **0.8%**	**0.852** / **0.0%**	88 min, 24 GB GPU
USM, whole library	**0.948–0.956** / 0–0.3%*	0.932 / 0.3% (trained)	0.646 / 36.9%	1.69 h, 48 GB GPU†

* Shards 0–2 only. † As reported by USM.

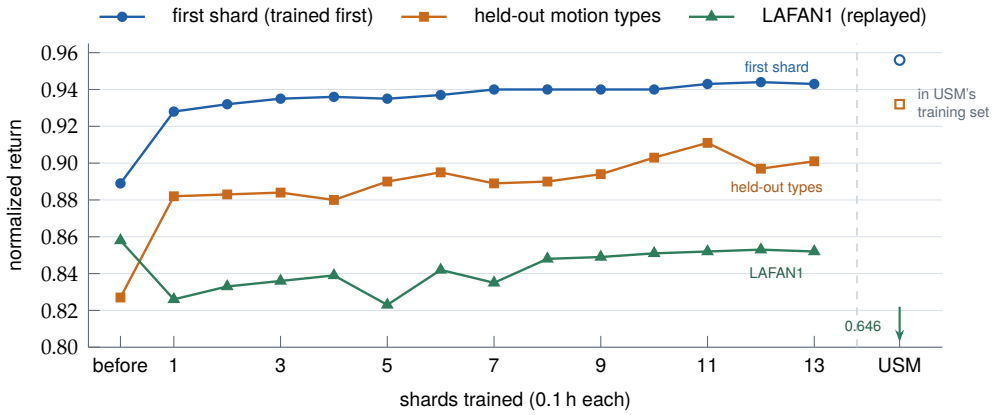


Figure 9. Sequential training on 13 shards of BONES-SEED, each warm-started from the previous one. The first shard's sample keeps improving after later shards replace its data, held-out motion types rise from 0.827 to 0.901, and LAFAN1, replayed three times in every shard, ends at 0.852 (0.858 before). Hollow markers: USM's tracker trained on the whole library at once [35]; its held-out-type score is on motions it was trained on, and it scores 0.646 on LAFAN1.

C.2 The stand-up acquisition

Table 7 gives the verification of request 3 (Section 5.2). The registry's plans also had their worst one-second tracking error fall from 14.5 to 9.8 cm. The factory replayed 4,998 library clips, about 4% of the library, whereas sharded training used about 9,770 per shard. An earlier attempt at request 1, which stopped before execution, cost $0.007; with it, the four requests of Table 1 cost $0.055 in API

calls. Acquisition also has a cost on borderline motions outside the request: on the other held-out types, failed rollouts rose from 1 to 5 of 1,698 while mean return rose from 0.934 to 0.936, all on a 360° flip and a spider crawl that also flipped between success and failure from shard to shard.

Table 7. Acquiring `kneeling_stop_R` in request 3: T_{13} before and T_K after 6.6 min of warm-start training (normalized return, failed rollouts).

Evaluation set	In new training data	Before	After
Stand up from a kneel: 14 held-out performers, 84 clips	no	0.734, 6/168	**0.817, 0/168**
Stand up from a kneel: training performers, 40 clips	yes	0.690, 4/80	0.832, 0/80
First-shard sample, 303 clips	yes	0.943, 0	0.945, 0
Other held-out motion types, 849 clips	no	0.934, 1/1,698	0.936, 5/1,698
LAFAN1, 40 clips	yes	0.852, 0/160	0.857, 0/160
Registry plans in ORCHESTRA, 128 worlds	–	128/128	128/128

C.3 Was the training needed?

Table 8 reruns the executed plan of every acquisition that trained and whose skills are still registered, with the tracker from before and after the training, and runs plans of the same shape with other clips of each type from held-out performers. Both trackers complete the same plans; training changes how precisely they are tracked. The factory trains when the probe's mean return is below 0.85 or a probe rollout fails; four of the five types had no failed probe rollout. The stops on other stand-up clips depend on the clip's first frame more than on the tracker: a plan that starts kneeling depends on how the robot is placed. For request 3, the registered clip already scored 0.897 with T_{13}, and the largest one-second body error of the executed plan was 11.2 cm before training and 12.9 cm after. Of the 66 stops of the two trackers on the 28 other stand-up clips, 65 came in the first 0.4 s, while the robot held the kneeling pose it was placed in; during the stand-up itself, T_{13} stopped once and T_K never. For the four training steps of chain B, both trackers also completed equally many plans built from other clips of the same types, and training lowered the body and heading errors of all four executed plans, most for the locking sequence, whose largest one-second body error fell from 19.2 to 6.2 cm.

Table 8. Was the training needed? *Executed plan*: the plan the loop executed, rerun in the same randomized worlds with the tracker before and after the training (3 seeds × 32 worlds); errors are those of the skill step, the p95 averaged over runs and the one-second maxima the largest over them. *Other clips*: plans of the same shape with other clips of the type from held-out performers, 16 worlds per clip (28 clips for request 3, 3–4 for the others). Pairs give before, after.

		Executed plan				Other clips
Acquisition	Probe before	Completed	Body p95 (cm)	1 s body max (cm)	1 s heading max (°)	Completed
Request 3, stand up from a kneel	0.670, 9/80	96/96, 96/96	10.9 → 10.9	11.2 → 12.9	10.7 → 9.9	415/448, 415/448
Chain B step 1, indiana step	0.837, 0/40	96/96, 96/96	7.6 → 6.1	7.7 → 5.3	6.2 → 3.7	64/64, 64/64*
Chain B step 2, locking down	0.741, 0/38	96/96, 96/96	10.3 → 7.1	19.2 → 6.2	10.2 → 6.3	45/48, 45/48
Chain B step 4, catwalk variation	0.760, 0/40	96/96, 96/96	6.7 → 6.5	7.5 → 6.3	7.0 → 4.6	64/64, 64/64
Chain B step 5, kick it	0.697, 0/40	96/96, 96/96	9.9 → 8.9	11.2 → 8.7	8.4 → 4.5	64/64, 64/64

* These four clips were in the training shards of both trackers; every other clip of the column was new to both.

C.4 Sequences of requests

Table 9 gives two sequences of five requests from T_K. Chain B is the sequence of Section 5.3. Chain A ran before it with the same requests and seed, and replayed the original training data and LAFAN1 but not the earlier acquisitions. In chain A every registered skill still executed at the end (256 of 256 worlds), but motion types learned early lost 0.02–0.06 in return, and failed rollouts on the other held-out types rose from 8 to 25; in chain B, early types lost 0–0.016 and those failures stayed between 5 and 8. LAFAN1 stayed at 0.86–0.88 and the first shard's sample at 0.94–0.95 in both chains, and replaying the kneeling motions acquired earlier gave chain B a better start on the locking sequence, which ends in a kneel: 0.739 before training, against 0.656 in chain A. The chains also differ in planner choices in steps 3 and 4 and in the amount of training, six training runs in chain A and four in chain B. The controlled comparison of Section 5.3 isolates replay: it attributes to replay the retention of earlier acquisitions, but not the difference in failures on unrelated types. The planner tended to reach for motions the tracker nearly performed: in both chains it served step 1 with a related motion (probe return 0.837) rather than the power step the request named, and in step 3 chain A's planner spent two request rounds on jumps the tracker already performed before requesting a motion that trained but yielded no clip fit for registration, while chain B's planner composed two directly registered jumps. Chain A's 8 registered skills were moved out of the registry when chain B ran. In chain B, the registry's plans completed in 128 of 128 worlds after every training. Both chains ran before the promotion gate existed: each request continued from the tracker that the previous one produced.

Table 9. Two sequences of five requests from T_K, each request starting from the tracker the previous one produced. Chain A replays the original training data and LAFAN1; chain B also replays the clips of all earlier acquisitions. "New type" is the return of the trained motion type on its held-out performers: before the step $\rightarrow$ after it $\rightarrow$ at the end of the chain. One seed.

			New type: before $\rightarrow$ after $\rightarrow$ end		Latency (min)	
Step	Request (abridged)	Motion type trained	Chain A	Chain B	A	B
1	quick hip-hop power step	indiana step	0.853 $\rightarrow$ 0.851 $\rightarrow$ 0.833	0.853 $\rightarrow$ 0.847 $\rightarrow$ 0.851	11.8	11.2
2	fast locking down into a kneel	locking down	0.656 $\rightarrow$ 0.807 $\rightarrow$ 0.769	0.739 $\rightarrow$ 0.787 $\rightarrow$ 0.788	11.7	11.2
3	jump in place, arms raised	A: jump around; B: none	not executed	no training	–	1.6
4	quick vogue catwalk	catwalk variation*	0.778 $\rightarrow$ 0.777 $\rightarrow$ 0.745	0.776 $\rightarrow$ 0.775 $\rightarrow$ 0.759	23.1	11.2
5	hip-hop kicking and punching	kick it	0.765 $\rightarrow$ 0.820 $\rightarrow$ 0.823	0.779 $\rightarrow$ 0.806 $\rightarrow$ 0.805	11.7	11.3
Requests served			4 of 5	5 of 5		
Other held-out types: failed rollouts after each training			8, 6, 9, 15, 19, 25	8, 5, 8, 5		
Last tracker on all plans of the chain and the registry			256/256	288/288		
API cost			$0.061	$0.062		

* Chain A also trained the plain catwalk in this step (A_4 before A'_4).

C.5 Controlled replay

Table 10 gives the controlled replay study of Section 5.3. Every chain starts from T_K and trains, in order, the indiana step, the locking sequence, jump around, the catwalk variation, and kick it, 2,000 iterations each, with a replay of 4,998 library clips and three copies of LAFAN1. The replay condition adds the training clips of every earlier acquisition (322 at step 1, 370 at step 5); the other condition adds as many random library clips. The five chain types have one held-out performer each (2–4 clips, 8 rollouts per clip); standing up from a kneel has 84 held-out clips. At the end of every chain, the first shard's sample scored 0.943–0.949 and LAFAN1 0.863–0.876, without failures. Over the five steps, the unrelated held-out types had 34–58 failed rollouts per chain without replay and 33–52 with it. Replay cost part of the gain on the new type at two of the five steps (0.053 against 0.023, and 0.052 against 0.037). The uncontrolled pair of request sequences above first suggested this design.

Table 10. Controlled replay: three training seeds per condition. Stand-up: return of the earliest acquisition after each step (start 0.817). Forgetting: return of each earlier type right after its own step minus at the end, averaged over the five earlier types. Unrelated failures: failed rollouts on 822 clips of held-out types that no step trains on (1,644 rollouts; 4 at the start), after steps 1–5.

		Stand-up after step						
Condition	Seed	1	2	3	4	5	Forgetting	Unrelated failures
No replay	0	0.822	0.814	0.809	0.766	0.755	0.030	13, 8, 6, 11, 9
No replay	1	0.808	0.814	0.801	0.771	0.800	0.007	8, 5, 5, 4, 12
No replay	2	0.804	0.805	0.803	0.779	0.722	0.049	16, 8, 15, 7, 12
Replay	0	0.814	0.829	0.798	0.828	0.833	−0.004	12, 4, 13, 4, 14
Replay	1	0.818	0.821	0.830	0.826	0.826	0.012	7, 7, 8, 6, 5
Replay	2	0.813	0.786	0.824	0.821	0.826	−0.004	10, 8, 11, 12, 11

C.6 Gated promotion: the decision record

Table 11 summarizes the promotion log of the gated sequence of Section 5.3, which ran on 2026-11-08 in a copy of the code, plans, and registry, starting from T_V; the project's registry pointer did not change. Every candidate passed the checks for resets and state writes, for the regression sets, and for the new motion type. The rejected candidate failed two plan checks, the meditation skill's plan and all plans together, while on the held-out clips of all earlier acquisitions, meditation's among them, its return rose from 0.831 to 0.836 with no failed rollout. The skipping skill was withheld, nothing was rolled back, and the copy's log holds five events. A first attempt at the sequence was discarded: its copy lacked the records of earlier acquisitions, so the gate could not run its check on them and rejected the first candidate, as it is designed to.

Table 11. The gated sequence of seven requests, from the promotion log. Trained: the motion type the request trained and its probe return with the starting tracker. Candidate: training run `m_<type>_s0_u7b_r`*i* and its final checkpoint. Checks: passed of run; their number grows with the registry. Gate: seconds from the executed plan to the decision, and in parentheses the part spent on the current tracker, whose cached results were reused from request 2 on. To plan, total: seconds from the request to the executed plan and to the end of the request. Pointer after: the tracker that the next request starts from; the first request starts from T_V. One run; API cost $0.10.

#	Request	Trained (probe)	Candidate	Checks	Decision	Gate	To plan	Total	Pointer after
1	Crawl forward on all fours like a spider.	spider crawl (0.698)	r1 / 2027	51/51	promoted	698 (338)	695	1,393	r1
2	Get down into a push-up position.	push-up start (0.811)	r2 / 1886	52/52	promoted	394 (46)	664	1,058	r2
3	Do some shadow boxing.	shadow boxing (0.839)	r3 / 1969	53/53	promoted	397 (47)	691	1,087	r3
4	Sit down cross-legged and meditate.	meditation (0.623)	r4 / 1958	54/54	promoted	408 (58)	685	1,093	r4
5	Skip forward.	skipping (0.734)	r5 / 1969	53/55	rejected*	397 (47)	676	1,072	r4
6	Jump around energetically, like at a concert.	none	–	–	–	–	31	31	r4
7	Do some shadow boxing, then get down into a push-up position.	none	–	–	–	–	27	27	r4

* Failed checks: the meditation skill's plan, 28/32 worlds with the current tracker against 0/32 with the candidate; all plans together, 1,532 against 1,504 of 1,536 worlds. Request 6 was served with registered jumps and a dance, request 7 with the skills of requests 2 and 3.

C.7 Composition and execution

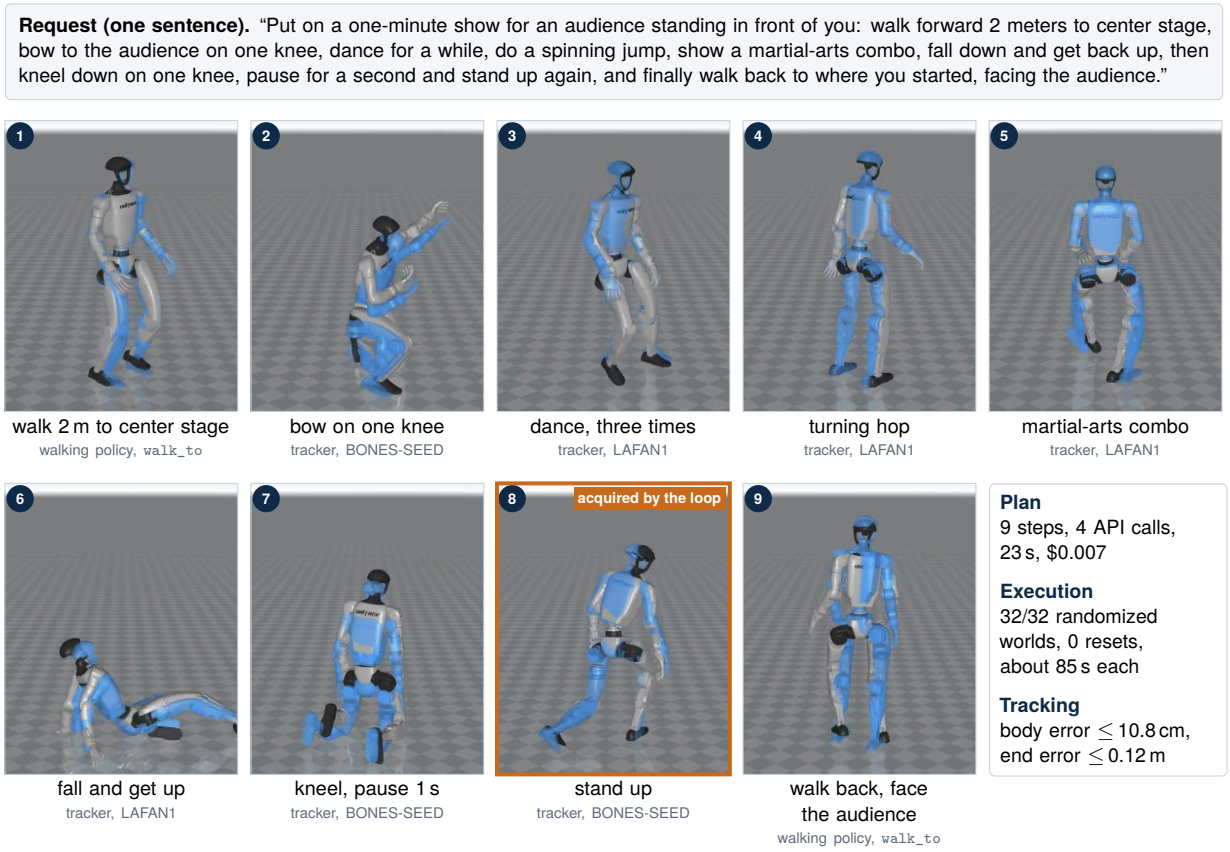


Figure 10. One request, one continuous simulation. The planner turned the request into a nine-step plan of registered skills and closed-loop walking, and ORCHESTRA executed it without resets in 32 of 32 randomized worlds. Frames show world 0 (gray: simulated robot; blue: reference motion). Step 8 uses a skill that the factory acquired for an earlier request, training the tracker for 6.6 min after the motion type failed the probe (Section 5.2).

Table 12 summarizes the composition tests of Section 5.4. The tracking error that a transition adds in its first second has a maximum of 5.3 cm, and all 71 pairs with a compatibility gap of at most 3 add less than 2 cm. Handoffs return control to the tracker with heading errors of at most 2.5°. With a BlinkMimic tracker, 8,191 of the 8,192 transitions completed. The single monitor stop occurred during a fall-and-get-up skill, rather than at a transition. Closed-loop `walk_to` ended a median of 0.05–0.11 m from its target, against 0.22–1.93 m open loop, and an injected fall stopped all 128 worlds at the step where it was injected. The 60 planner sessions cost $0.198, with 8 worlds per plan; most failed checks came from "Run forward a bit.", for which the registry had no run that stops, and once such a skill was registered, the pass rate on the two affected requests rose from 5/12 to 9/12. In the nine-step plan of Figure 10, the largest one-second average body error was 10.8 cm, `walk_to` ended at most 0.12 m from its targets, and the planner stated two approximations in the plan: the spinning jump is a small hop with a 100° turn, and the dance repeats one short clip three times.

Table 12. Composition and execution of motion skills. Counts are worlds or trials that completed out of those run.

Test	Setting	Result
Skill transitions	16 skills, 256 ordered pairs × 32 worlds; T_0 with and without blending, BlinkMimic	8,192, 8,192, 8,191 of 8,192
Tracker–walking handoff	4 plans, 2 trackers, nominal and randomized	448/448 completed, 0 falls
Closed-loop `walk_to`	7 tasks × 32 worlds	224/224 reached the target
Multi-skill plans	7 plans (3–16 steps, 16–100 s), 2 trackers, nominal and randomized	896/896 completed
Fault injection	fall injected at step 3	128/128 stopped at step 3
Language requests	10 requests × 3 runs × 2 effort levels; 8 worlds per plan	52/60 pass all checks; 424/424 executed
One-sentence performance	9 steps, about 85 s (Figure 10)	32/32 completed

C.8 The scaling grid

Table 13 gives the scaling grid of Section 6.1. The learning-rate schedule depends only on the iteration and the training return, so the state after b iterations is the tracker that a run of b iterations would produce. The 1k library after 32k iterations (0.840) stays below the 3k library after 4k (0.856). At equal wall-clock time, the larger library wins up to 10k clips: after about 1.4 h, the 1k library reaches 0.840 and the 10k library 0.891. Without the training-time evaluation, an iteration runs at about 8.5 per second; moving that evaluation to five additional GPUs also lowers the training GPU's memory from 60.6 to 39.2 GiB, although loading the library still peaks at 55.9 GiB.

Table 13. Zero-shot return on 112 held-out motion types (1,000 clips, 2 rollouts each) for libraries of increasing size, each trained from scratch. In parentheses: held-out types usable without training (no failed rollout, mean return ≥ 0.85). Training: wall-clock time on one A800 for 32k iterations.

Library	Types	Hours	4k iterations	8k	16k	32k	Training
1k clips	531	2.0	0.815 (83)	0.818 (81)	0.831 (84)	0.840 (85)	1.4 h
3k clips	1,066	6.3	0.856 (88)	0.855 (87)	0.868 (89)	0.882 (91)	1.7 h
10k clips	2,070	20.7	0.875 (88)	0.885 (90)	0.891 (91)	0.902 (95)	2.8 h
30k clips	3,129	61.5	0.882 (90)	0.889 (92)	0.893 (94)	**0.908** (97)	5.5 h
60k clips	3,353	122.7	**0.891** (91)	**0.891** (92)	**0.902** (95)	**0.908** (96)	9.8 h

C.9 Motion types and clips

Table 14 gives the two diversity tests of Section 6.1. In the first, each library holds 10k clips from the 13 training shards: two draw 500 and 1,000 motion types at random and then 10k clips from their clips (the 1,000 drawn types put 935 into the library), one is the 10k library of the scaling grid, and one takes one clip of every type and the rest at random. The second keeps the 1,000 types of that draw, all present, and takes one clip of every type and the rest at random, to 3k, 10k, and 30k clips. Each library was trained from scratch as in the grid. On the 30 new types of Section 6.2, return before fine-tuning rises from 0.891 with 500 types to 0.899–0.909 with more, and is 0.890, 0.904, and 0.907 for the 1,000 types with 3k, 10k, and 30k clips. The four libraries of 10k clips hold 18.6–21.4 h of motion. From 500 to 935 types, failed rollouts on the held-out types fall from 104 to 17.5 of 2,000;

beyond that, the return stays at 0.900, with seeds within 0.007 of each other, and only the number of usable held-out types still rises a little, from 90.5 to 95 of 112. The 1,000 types with 3k, 10k, and 30k clips hold 6.0, 18.5, and 54.7 h of motion; from 10k to 30k clips, return gains 0.003 for twice the training time (2.6 against 5.2 h on an A800), and 1,000 types in 3k clips beat 500 types in 10k clips with less than a third of the clips and of the motion hours.

Table 14. Motion types and clips: zero-shot results on the 112 held-out types (1,000 clips, 2 rollouts each) after 32k iterations, mean of two training seeds. Training: wall-clock time to 32k iterations on one A800.

Library	Types	Clips	Hours	Return	Failed (of 2,000)	Usable (of 112)	Training
Fixed number of clips, types varied							
500 types drawn at random	500	10k	20.3	0.851	104	90.5	2.6–2.7 h
1,000 types drawn at random	935	10k	18.6	0.900	17.5	90.5	2.6–2.7 h
Scaling grid, 10k clips	2,070	10k	20.7	0.900	8.5	93.5	2.6–2.7 h
Every type at least once	3,376	10k	21.4	0.900	14	95	2.6–2.7 h
Fixed types, clips varied							
1,000 types, every one present	1,000	3k	6.0	0.884	20.5	88	1.7 h
1,000 types, every one present	1,000	10k	18.5	0.902	15	91.5	2.6 h
1,000 types, every one present	1,000	30k	54.7	0.905	11.5	92.5	5.2 h

C.10 Fits of the scaling grid

Figure 11 shows the fits of Section 6.1. A joint fit of the remaining error in the number of motion types T and iterations I, $1 - R \propto T^{-0.28} I^{-0.10}$, deviates from the 20 grid points by at most 7.5%, against 12% for the same fit in the number of clips; fitted on the 1k–30k libraries, it predicts the four 60k points with a mean error of 0.002, while the clip-based fit overestimates them by 0.011. The power law in types describes the grid, where types and clips grow together; it is not a law of either axis alone.

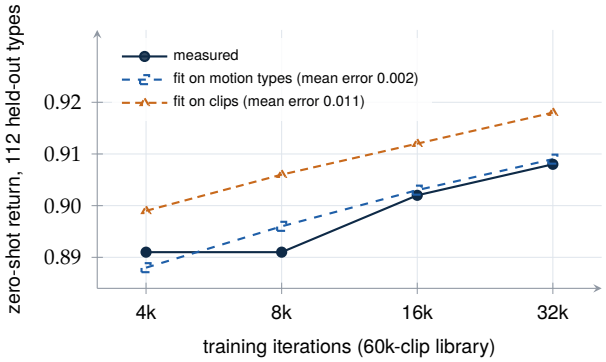
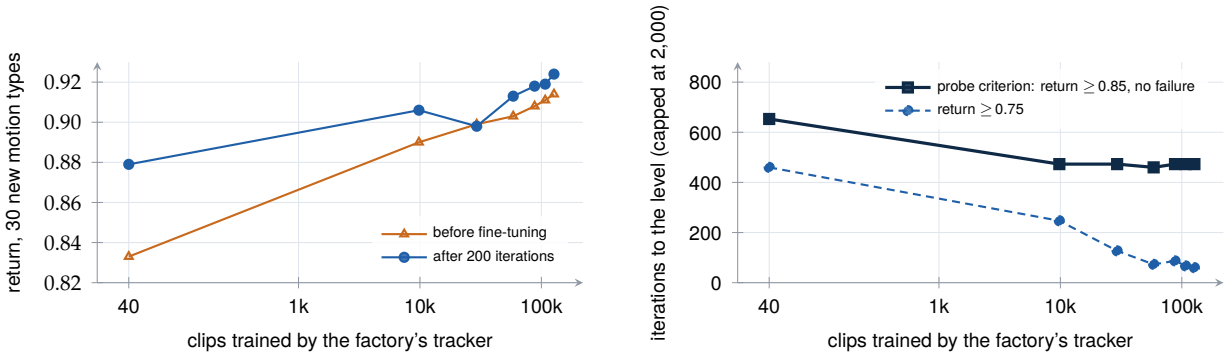


Figure 11. Fits of the scaling grid (one seed). Fits of the remaining error, 1 − return, on the 1k–30k libraries predict the 60k library accurately when they use the number of motion types and overestimate it when they use the number of clips; from 30k to 60k the clips double while the types grow by 7%.

C.11 Adaptation effort along the factory's history



(a) Return on the 30 new motion types.

(b) Fine-tuning iterations to a level.

Figure 12. Adaptation effort along the factory's history (one seed): the seven trackers from T_0 to T_{13}, fine-tuned without replay on 30 new motion types and evaluated on held-out performers. The history grows library and training together, so the comparison is observational. (a) Return before fine-tuning and after 200 iterations. (b) Mean fine-tuning iterations after which a type stays at a level without failed rollouts; a type that starts there counts 0, and one that does not reach the level within 2,000 iterations counts 2,000. From T_1 on, 6–7 of the 30 types do not reach the probe criterion within 2,000 iterations, and their capped values make up most of its mean.

In the grid, return on the 30 new types before fine-tuning is 0.858, 0.893, 0.902, 0.911, and 0.919 for the five libraries after 32k iterations; after 200 fine-tuning iterations it rises from 0.87 to 0.89–0.91 between 1k and 10k clips and then stays at 0.905–0.909; on the 5 hard types it is 0.65–0.67 for the 1k library and 0.72–0.80 for all larger ones. The seven trackers of the factory's history span 0.10 to 1.38 h of cumulative training; their return on the 30 types before fine-tuning rises from 0.833 to 0.914, and after 200 iterations from 0.879 to 0.924 (Figure 12a). As a proxy for adaptation cost, we count the fine-tuning iterations after which a type reaches and keeps a return of 0.75 without failed rollouts (Figure 12b). This proxy falls from 460 to 60 iterations on average, and from 1,480 to 360 on the 5 hard types, though not at every step, and of the 112 held-out types, the number usable without any training rises from 79 to 97. Because this fine-tuning trains on the new types alone, an iteration costs about the same for every starting tracker, so fewer iterations also mean less GPU time per new type; the cost of building the larger library, 1.38 GPU-hours of cumulative training for T_{13}, is shared by all later acquisitions and not included. At a return of 0.80, the mean cost falls from 547 to 273 iterations (Table 15). The proxy is therefore evidence that basic adaptation gets cheaper, not a measurement of what an acquisition costs. The history also grows the library and the training together; the grid shows that both contribute.

Caveats. Of the 30 new types, 25 have a single held-out performer, so individual cells are noisy and the conclusions rest on averages and trends. Fine-tuning without replay degrades the grid's larger trackers over time (7 of the 12 points with at least 10k clips score 0.01–0.03 lower after 2,000 iterations than after 200) but not the factory's trackers, which were trained by sequential warm starts; we have not identified the cause. The factory itself always fine-tunes with replay.

C.12 Learning cost at three levels

Table 15 reads the fine-tuning curves of Section 6.2 at three levels, without new training. At the probe criterion, from T_1 on, 23 of the 30 types reach it within 200 iterations and at most one more

within 2,000; of the five hard types, only one or two reach it within 2,000 iterations with any tracker. Usable without training here counts the types whose held-out performers, up to 24 clips, meet the probe criterion at iteration 0; the factory's probe in Table 2 uses up to 40 clips from all performers, which explains 21 against 19 of 30 types from T_0. In the scaling grid, the cost at the probe criterion shows little trend beyond 1k clips: 733–800 iterations at 1k clips and 473–733 from 3k to 60k, with 7–12 of the 30 types not reached.

Table 15. Fine-tuning iterations that the 30 new motion types need along the factory's history, at three levels: the mean over types, where a type that does not reach the level within 2,000 iterations counts 2,000, and in parentheses the number of such types. The column ≥ 0.85 is the probe criterion; the last column gives the 5 hard types at a return of 0.75. Usable at once: usable without any training.

			All 30 types, return			5 hard types
Tracker	Clips trained	Usable at once	≥ 0.75	≥ 0.80	≥ 0.85, no failure	≥ 0.75
T_0	40	21/30	460 (4)	547 (7)	653 (9)	1,480 (3)
T_1	9,806	22/30	247 (3)	473 (6)	473 (7)	1,440 (3)
T_3	29,335	22/30	127 (1)	460 (6)	473 (7)	720 (1)
T_6	58,625	23/30	73 (0)	327 (4)	460 (6)	440 (0)
T_9	87,923	23/30	87 (1)	340 (5)	473 (7)	520 (1)
T_{11}	107,458	23/30	67 (0)	307 (3)	473 (7)	400 (0)
T_{13}	126,984	23/30	60 (0)	273 (3)	473 (7)	360 (0)

C.13 The cost ledger

Table 16 summarizes the factory's own records of every acquisition made on the RTX 4090 before the promotion gate existed; Table 3 adds the gated sequence. From request to executed plan, a request took 45–97 s without training, 671–709 s with one training, 1,389 s with two, and 76 and 101 min for the two interaction skills.

Table 16. The cost ledger: 29 acquisitions for 16 requests, from recorded wall-clock timings. The 16 requests cost $0.22 in API calls.

Path	Acquisitions	Median time to a validated skill	Median GPU time	Registered
Usable at once: probe, register, validate	15	20 s	19 s	15
Trained: probe 9 s, train 410 s, evaluate 238 s, register 0 s, validate 9 s	12	666 s	662 s	11
Interaction skill, staged training	2	4,547 and 6,079 s	4,428 and 5,977 s	1

C.14 Skills from video

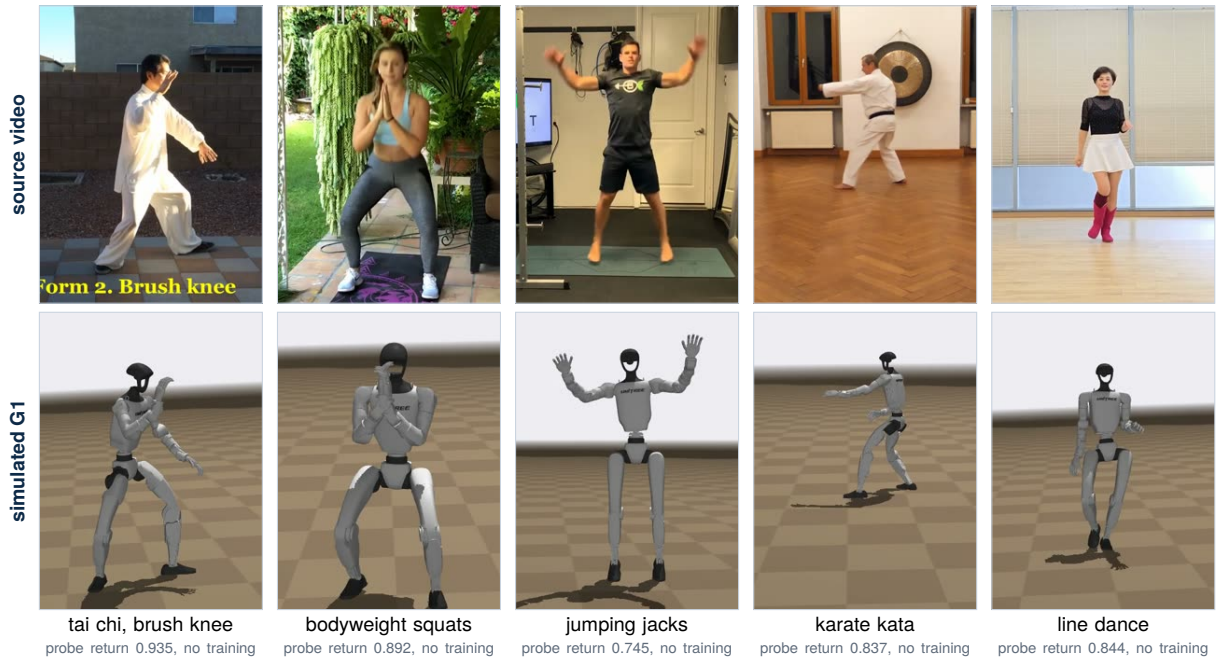


Figure 13. Skills from web videos. Each column pairs a frame of a Creative Commons video (top) with the simulated G1 executing the registered skill made from it, in ORCHESTRA world 0 of 32, rendered from the same camera (bottom). T_V performed all five motions without training. Videos: "8 Form Tai Chi (Demo)", Tai Chi iStudio; "Bodyweight Squat", Get Strong with Emily; "Jumping Jacks", Paul Sandburg Fitness; "Shotokan Karate Kata Heian Shodan", Christoph Jantzen – Kampfkunst Lausitz; "Love Potion 666 Line Dance", May Cho Linedance. All are published on YouTube under the Creative Commons Attribution license.

Table 17 gives the skills from video of Section 7.2. Of the 6–8 min from a video's URL to a validated skill, 1.8–3.1 min went to motion estimation, 3.2–4.2 min to optional comparison videos for human review, and about 30 s to the factory. The jumping jacks lose most of their leg spread while the arms still go fully overhead; squat depth and the line-dance steps are reproduced.

Table 17. Skills from web videos. *Feasibility*: the factory's probe (4 rollouts, none fell; the motion is new to the tracker) and the registered skill alone in ORCHESTRA (32 randomized worlds), with the body position error during the skill (mean / 95th percentile). *Fidelity*: median horizontal distance between the ankles in the retargeted reference → in simulation (world 0), compared over the skill window. Time: from the video URL to a validated skill and its rendered clip. The first row was probed with B_5 at the standard level of 0.85, the others with T_V at 0.7.

	Feasibility				Fidelity	
Skill	Probe	Factory decision	Execution	Body error	Feet apart	Time
Dribble dance	0.774	trained 0.1 h (0.766 after)	32/32	6.1 / 11.2 cm	–	–
Tai chi, brush knee	0.935	registered	32/32	3.7 / 5.5 cm	0.58 → 0.43 m	7.7 min
Bodyweight squats	0.892	registered	32/32	3.3 / 4.9 cm	0.37 → 0.36 m	8.1 min
Jumping jacks	0.745	registered	32/32	6.9 / 9.5 cm	0.29 → 0.28 m*	6.2 min
Karate kata	0.837	registered	32/32	5.6 / 9.3 cm	0.60 → 0.46 m	7.0 min
Line dance	0.844	registered	32/32	4.2 / 6.5 cm	0.17 → 0.17 m	6.8 min

* The median hides the loss: the legs open and close over 0.24–0.35 m instead of 0.10–0.61 m (5th to 95th percentile).

C.15 Transfer to other engines

Table 18 gives the transfer results of Section 7.3. Evaluation reimplements InstantTrack's return and fall test outside the training code; on 32 clips it matches InstantTrack's own evaluation to within 8×10^{-7} in return, with identical fall decisions, and the ONNX policy matches the training network to within 2×10^{-6}. In classic MuJoCo no fall decision flips, also with an explicit PD loop at 1 kHz. The one held-out clip that fails everywhere is a 360° flip, which also fails in training, and Isaac Sim fails one LAFAN1 clip 205 s into its 225 s. Genesis adds failures only on motions in which the knees and shins carry the robot, where the two engines treat mesh contact differently.

Table 18. The exported tracker B_5, without retraining, in four physics engines and under two perturbations. Each clip is played once from its first reference frame, without randomization or noise. Return (failed clips / clips). Held-out types: 108 clips of motion types that the tracker never trained on.

Simulator or setting	Held-out types	Registry skills	LAFAN1
Training simulator (MuJoCo Warp)	0.926 (1/108)	0.926 (0/32)	0.866 (0/40)
MuJoCo, CPU	0.925 (1/108)	0.926 (0/32)	0.865 (0/40)
MuJoCo, CPU, explicit PD at 1 kHz	0.927 (1/108)	0.926 (0/32)	0.865 (0/40)
Genesis [7]	0.926 (1/108)	0.899 (1/32)	0.864 (0/40)
Isaac Sim (PhysX), via Isaac Lab [39]	0.920 (1/108)	0.916 (0/32)	0.846 (1/40)
MuJoCo, CPU, 20 ms action delay	0.900 (1/108)	0.882 (0/32)	0.788 (2/40)
MuJoCo, CPU, ground friction 0.5	0.917 (1/108)	0.905 (0/32)	0.730 (11/40)

C.16 The benchmark, request by request

Table 19 gives the results of rsf-bench v0 (Section 5.1) by category and Table 20 by request. Each request that trained was rerun with the tracker from before its training, in the same 32 worlds. The training was needed for the plan to complete in three cases: in S1, the spider crawl (10 of 32 worlds before the training, 32 after); in S2, the push-up position (25 of 32) and skipping (22 of 32). It was not needed for the push-up position, shadow boxing, skipping, and D7 in S1, nor for the spider

crawl, the sprint, and shadow boxing in S2, all of which completed in 32 of 32 worlds before and after. The run had no semantic check of the executed motion: the components rule can pass a motion that only partly fits the request and fail an honest approximation. D5 took 53 s in S1 and 25 s in S2. In S2, two new types completed in fewer worlds with the accumulated tracker before training than with the base tracker (25 and 22 of 32, against 32 of 32), an observation from one run that repeated runs need to check. Besides the planner's approximations, one request crashed on a motion type with a single performer, a bug fixed since, and two honest approximations, a drinking motion without the cup and a hands-and-knees crawl for a spider crawl, fail the benchmark's mechanical rules. The benchmark ran before the promotion gate existed.

Table 19. rsf-bench v0: requests accepted, by category, one run per system from the same base tracker. B and C are labeled by the probe of the base tracker. Table 20 gives every request.

Category	Requests	S0 registry only	S1 factory, fresh start	S2 factory, accumulating
A an existing skill	6	6	6	6
B a new type that passes the probe	6	0	4	4
C a new type that fails the probe	6	0	4	3
D a composition	7	3	6	6
E walking and interaction experts	5	5	5	5
F should be declined	7	7	6	6
Accepted, requests that call for execution (A–E)	30	14	25	24
Accepted, requests to decline (F)	7	7	6	6
Accepted, all requests	37	21	**31**	**30**
Acquisitions (of them trained)		0	16 (6)	13 (5)
API cost		$0.22	$0.42	$0.43

Table 20. rsf-bench v0 by request. Acquisitions and training runs are counted per request; times run from the request to the executed plan.

Requests	Cat.	S0	S1: factory, fresh start	S2: factory, accumulating
A1–A6	A	6/6	6/6, no acquisition	6/6; for A5 (sprint) the planner trained a new run type (678 s) although a sprint skill was registered
B1, B2, B3, B6	B	no	yes; 1 acquisition each, no training, 43–50 s	the same
B4 (stadium cheer), B5 (jump for joy)	B	no	no: approximated with registered jumps and waves	no, the same
C1 (spider crawl)	C	no	yes; 3 acquisitions, 2 training runs, 1,378 s	no: a hands-and-knees crawl; 3 acquisitions, 1 training, 1,380 s
C2 (push-up position), C4 (shadow boxing), C6 (skipping)	C	no	yes; 1 training each, 665–682 s	yes; 1 training each, 669–684 s
C3 (jump around)	C	no	no: approximated with registered dances and jumps	no, the same
C5 (meditate)	C	no	crashed on a type with a single performer	the same
D1–D4	D	D1–D3	yes	yes
D5 (hip hop, then bow)	D	no	yes; acquired the bow again, 53 s	yes; reused it, 25 s
D6 (clap, then jump for joy)	D	no	no: a simple jump, 53 s	no, the same, 24 s
D7 (lunge, then push-up position)	D	no	yes; acquired both again and retrained the push-up, 688 s	yes; reused both, 27 s
E1–E5	E	5/5	5/5	5/5
F1, F3–F7	F	yes	yes	yes
F2 (drink from a cup)	F	yes	no: a drinking motion, with the pick-up left out	no, the same

C.17 Design choices

Table 21. Effect of individual design choices. Each row compares the system with one variant; the last column names the experiment the setting comes from.

Design choice	Measure	With	Without	Setting
Warm start from the previous tracker (vs. from scratch)	training time until the tracker stays usable ↓	**2.2–2.9 min**	≥ 5.0 min	LAFAN1 study
LAFAN1 replayed 3 times per shard (vs. once)	LAFAN1 after 3 shards: return, failures	**0.836, 0/160**	0.81, 2–5/160	Section 4
Earlier acquisitions in the replay (vs. as many random clips)	mean forgetting of earlier acquisitions, return ↓	**0.001**	0.029	Section 5.3
Heading-aligned reference at a skill switch	falls; body error 1 s after the switch ↓	**0/64; 3.2 cm**	1/64; 9 cm	64 worlds
Closed-loop `walk_to` (vs. open-loop walking)	median end error: position; heading ↓	**0.05–0.11 m; 2–5°**	0.22–1.93 m; 11–27°	Section 5.4
Calibrated walking model in `check_plan`	worst predicted end-position error ↓	**0.46 m**	0.75 m	planner evaluation
Hand-off training of the box policy	walk–carry–return plans completed ↑	**32/32**	0/32	Section 7.1
Hand collision geometry in ORCHESTRA's G1	standalone box success ↑	**24/32**	17/32	Section 7.1
Backing off after an interaction skill	second carry of a relay succeeds ↑	**32/32**	4–8/32	Section 7.1

Table 21 isolates nine design choices. The first row comes from a controlled acquisition study on LAFAN1, in which we held out 5 of the 40 clips, trained a tracker on the other 35, and treated the 5 as new motions. New performances of known motion types were usable without training, which is the case the factory's probe registers directly. A new motion type, three jumping clips, failed in 2, 4, and 4 of 4 rollouts; warm-starting the tracker on all 40 clips for 0.1 h removed every failure, raised the jumps to 0.84–0.86, and moved the 35 earlier clips from 0.858 to 0.869, and import to a usable registered skill took 10.6 min (two seeds). Warm-starting makes the tracker usable after about half the training that training from scratch needs, which is what keeps an acquisition with training near ten minutes.

Replay protects old data at two scales: three copies of LAFAN1 per shard keep LAFAN1 at 0.836 after three shards, against 0.81 with one copy, and replaying clips from earlier acquisitions reduces forgetting during subsequent updates. Aligning the reference with the robot's heading at a switch lowers the tracking error after the switch from about 9 to 3.2 cm. Closing the loop on position lowers the median end error of walking from 0.22–1.93 m to 0.05–0.11 m across the same seven tasks, and calibrating the planner's walking model lowers its worst prediction error from 0.75 to 0.46 m. The last three rows concern handoffs between experts: an interaction policy must be trained on the state in which it receives the robot, the simulated robot must match the one it was trained on, and the walking policy must leave room before it turns.

C.18 Interaction experts: results

Table 22 gives the box-carrying results of Section 7.1. Alone on the box reference, the tracker lifts the box above 20 cm in only one world. The first box policy trained on its clip for 6,000 iterations

(87 min); the hand-off fine-tuning ran 3,000 iterations (53 min) on episodes that partly start from the walking policy's stance, on a reference that ends with 1 s at rest, and the hand-off policy also succeeded when the same box was carried twice. The four large-turn clips carry the box 1.9–2.4 m and were fine-tuned from the hand-off policy for 6,000 iterations each. Two different box skills chain within one run when the walking policy backs off after each skill: with the back-off, all 32 second carries succeed, and without it only 4–8 do. Before the same hand-over fine-tuning, a climbing step-over whose clip starts in motion succeeded in 1 of 32 worlds and a wall vault in 10 of 32.

Interaction requests. Asked to move a box about 1.5 m to the right and walk back, the planner chose a skill that carries the box back and to the right and rotated the box by $-90°$, which the near-cubic box allows; the plan succeeded in 28 of 32 worlds. The factory acquired a new box skill, a carry about 1.6 m to the left, from one library clip in 55 min, 50.5 of them training; it succeeded in 26 of 32 worlds. With this skill hidden, the planner searched the interaction library once and served a request for a left carry without requesting a skill, with the back-right carry and the box rotated by $+90°$ (24 of 32 succeeded, 109 s, \$0.024); without the factory tools, the same request ended without a plan. The gap was only apparent, and the planner found the symmetry that closes it. On the platform with a jump down, every attempt of the climbing loop fell in all 32 worlds.

Table 22. Box carrying, 32 randomized worlds per row. Every row except the first is an ORCHESTRA plan executed as one continuous run, with T_0 as the tracker; the first evaluates the tracker alone on the box reference. *Completed*: worlds that executed every step without a monitor stop. *Success*: the box ends within 10 cm of where the skill's reference puts it.

Plan or setting	Completed	Success	Box end error
Tracker alone on the box reference, box in the scene	32/32	0/32	0.67–0.68 m
Box policy without hand-off training: walk, carry, walk back	0/32	0/32	–
Hand-off-trained box policy: walk, carry, walk back	32/32	32/32	6.5 cm
Dance, carry, walk back, bow (Figure 6)	32/32	32/32	6.7 cm
Two carries in a row, the second started 22 cm off	32/32	32/32, 32/32	6.4, 4.8 cm
Four skills with turns of 113–210°, one plan each	$4 \times 32/32$	32, 26, 26, 30	4.6–7.6 cm
Relay of two different box skills, then walk back	32/32	32/32 (second)	–
Planner: carry 1.5 m right, by rotating the box for a back-right carry	32/32	28/32	7.1 cm
Factory: new box skill acquired from the library in 55 min	32/32	26/32	8.0 cm
Loop with the left-carry skill hidden: planner reuses the back-right carry	32/32	24/32	8.0 cm

C.19 Interaction experts: engineering notes

Three differences explained why the first box policy, which succeeded in 29 of 32 worlds alone, failed when the walking policy handed it the robot. ORCHESTRA's G1 model lacked collision geometry on the hands, which lowered standalone success from 24 to 17 of 32 on that model; the walking policy hands over a robot standing still in its own stance, from which the policy succeeded in 0 of 32 worlds; and the clip ends while the person is still walking toward the box. After the hand-over fine-tuning, the walking policy reached the skill's start pose within 7.5 cm on average (10.9 cm at most), and the box ended 4.8–6.7 cm from its target; the second of two carries started 22 cm off and still succeeded in all 32 worlds. The four large-turn skills succeeded in 25, 24, 3, and 0 of 32 worlds after 3,000 iterations and in 32, 26, 26, and 30 after 6,000; for the large turns, the remaining error

lies in where the robot itself ends up, which more training reduced from 16–21 cm to about 6 cm. These skills vary strongly between checkpoints: one fell from 26 to 7 of 32 when training continued to 9,000 iterations, and quadrupling the reward weights on global position improved standalone evaluation but lowered ORCHESTRA success, from 26 to 23 and from 30 to 14. We therefore select interaction checkpoints by their results in ORCHESTRA. Without the back-off step, the walking policy pushed the box 0.2–0.4 m while turning toward the next start pose.

C.20 Does the box policy need to observe the box?

At equal training iterations, a policy that observes the box and one that observes only proprioception, with the box visible to the critic alone, succeed in 12 and 11 of 32 worlds after 2,000 iterations. In this evaluation the box always starts where the reference expects it, so the reference already tells the policy where the box is; OmniRetarget [52] also carries boxes with policies that do not observe them, and HAIC [20] reports a proprioception-only baseline that does so on a real G1. Observation should matter once the box departs from the reference, for example after a bump, which we have not yet tested.

D Comparison with related systems

Table 23. Systems that acquire or compose robot skills, compared on the properties that a self-improvement loop needs. ✓ yes, ∼ partly, ✗ no or not reported, – not applicable because no shared controller changes. *Gap-triggered*: acquisition starts because the current skills cannot serve a task.

System	Platform	What a new skill changes	Gap-triggered	Growing library	Re-verifies earlier skills	Chains skills without resets
Humanoid systems with a frozen controller						
HomeBody [14]	G1 with hands; real, sim	hand-written skill module	✗	∼	–	✓
CHOREO [42]	G1; sim	trajectory asset for a frozen tracker	✗	✓	–	✓
InterEvolve [28]	G1; sim, real	reward program for a frozen controller	∼	✓	–	✓
Metatron [44]	G1; sim, real	goal program for a frozen policy	✗	✗	–	∼
Humanoid systems that update the controller						
Continual HML [13]	G1; sim, real	a new adapter column; earlier columns frozen	✗	✓	✓	✗
Gap-triggered acquisition in manipulation						
InSight [47]	robot arm; sim, real	the VLA, retrained on self-collected demonstrations and its original data	✓	✓	✓	✓
AdaRSI [48]	RoboCasa365 kitchens; sim	a sub-step policy fine-tuned from the VLA, used where it measurably helps	✓	✓	–	✓
Agentic skill learning in other domains						
ASPIRE [30]	arms, mobile manipulators; sim, real	program code and text skills	∼	✓	✗	∼
SCALAR [55]	Craftax (game)	per-skill policy head on a shared backbone	∼	✓	✗	✓
RoboGen [49]	arm, quadruped, soft bodies; sim	new policy per task	✗	✗	–	∼
RoboForge* [23]	Go2, designed robots; sim, real	new policy per task	✗	✗	–	✗
Ours	G1; sim	**the shared tracker, warm-started;** or a new interaction policy	✓	✓	✓	✓

* The agentic framework of Li et al. [23]. A different system of the same name [54] refines a G1 tracker offline on text-generated motions, without an agent. HML: humanoid motion learning.

E Data, licenses, and credits

- **Code.** Robot Skill Factory and ORCHESTRA: Apache-2.0. InstantTrack and BlinkMimic are MIT-licensed, and mjlab and MuJoCo Warp are Apache-2.0. The planner calls Claude Haiku 5.5 through Anthropic's API, behind a tool interface that another model can replace.

- **Trained weights** are published with the release, each under the terms of its training data and listed in the code repository's `WEIGHTS.md`: the G1 and H1-2 trackers are for non-commercial research (LAFAN1 is CC BY-NC-ND 4.0), the walking policy is Apache-2.0, and the interaction policies are MIT.
- **LAFAN1** [12], retargeted to the G1 and the H1-2 [33]: CC BY-NC-ND 4.0. Trackers trained on it are for non-commercial research.
- **BONES-SEED** [3]: academic non-commercial research or qualifying start-ups, no redistribution of the motion data. Training data of our BONES-SEED trackers includes Motion Data by Bones Studio (https://bones.studio/).
- **OmniRetarget** robot–object and robot–terrain data [52], derived in part from OMOMO [21]: MIT.
- **Videos.** The five videos of Figure 13 and Table 17 are published on YouTube under the Creative Commons Attribution license; their titles and channels are credited in the caption of Figure 13. The segments used are seconds 58–72 (tai chi), 2–15 (squats), 4–16 (jumping jacks), 8–22 (kata), and 18–31 (line dance). The first video of Table 17 is YouTube video `GLu5YwiAtC4`, seconds 20–34. Only the retargeted motions are used for training; the videos are not redistributed.
- **Tools.** GEM-X [37] and soma-retargeter [38], which do not use the SMPL family of body models: code under Apache-2.0, with per-file exceptions in GEM-X; the GEM-X weights are under the NVIDIA Open Model License and include a DINOv3 backbone, and the person detector was trained on data restricted to non-commercial use. Genesis [7]: Apache-2.0. Isaac Sim requires accepting NVIDIA's license agreement. Unitree's G1 and H1-2 descriptions: BSD-3-Clause.
- **Figures.** Figures 1 to 3 are vector illustrations made for this report; Figures 1 and 2 show Anthropic's Claude mark. The simulation frames in Figures 6, 10 and 13 show the simulated G1 tracking motions from LAFAN1, BONES-SEED, OmniRetarget, and the videos above.